

MÓDULO PROYECTO

CFGS Desarrollo de Aplicaciones Multiplataforma
Informática y Comunicaciones

APP CARRETILLERO

Tutor individual: Juan Antonio Alonso Velasco

Tutor colectivo: Cristina Silvan Pardo

Año: 2024/25

Fecha de presentación: 21-05-2025

Nombre y Apellidos: Carla de la Fuente Bernardino

Email: carla.fueber@educa.jcyl.es



ÍNDICE

1. IDENTIFICACIÓN DEL PROYECTO	3
2. ORGANIZACIÓN DE LA MEMORIA	3
3. DESCRIPCIÓN GENERAL DEL PROYECTO	4
3.1. Objetivos	4
3.2. Cuestiones metodológicas.....	4
3.3. Entorno de trabajo (tecnologías de desarrollo y herramientas)	5
4. DESCRIPCIÓN GENERAL DEL PRODUCTO	5
4.1. Visión general del sistema.....	6
4.2. Descripción de métodos, técnicas o arquitecturas utilizadas	7
4.3. Despliegue de la aplicación	7
5. PLANIFICACIÓN Y PRESUPUESTO.....	9
6. DOCUMENTACIÓN TÉCNICA	10
6.1. Especificación de requisitos	10
6.2. Análisis del sistema	10
6.3. Diseño del sistema	13
6.3.1. Diseño de la Base de Datos	13
6.3.2. Diseño de la Interfaz de usuario.....	16
6.3.3. Diseño de la Aplicación	19
6.4. Implementación	20
6.4.1. Entorno de desarrollo.....	20
6.4.2. Estructura del código	21
6.4.3. Cuestiones de diseño e implementación reseñables.....	38
6.5. Pruebas.....	39
7. MANUALES DE USUARIO	40
7.1. Manual de usuario	40
7.2. Manual de instalación.....	48
8. CONCLUSIONES Y POSIBLES AMPLIACIONES.....	48
9. BIBLIOGRAFÍA	50
10. ANEXOS	50
10.1. ANEXO I: Código de la aplicación	50
10.2. ANEXO II: Control de versiones	50
10.3. ANEXO III: Diagrama de Gantt	51

1. IDENTIFICACIÓN DEL PROYECTO

Este proyecto surge debido a la necesidad de desarrollar una aplicación que permita a los Jefes de Taller (JT) disponer de información actualizada sobre la saturación de los puestos de trabajo de sus operarios. Actualmente, estos datos se registran de manera manual en hojas de cálculo y documentos físicos, lo que dificulta la gestión eficiente y para el correcto funcionamiento de las plantas.

Esta problemática la detecté al incorporarme a la empresa Horse, donde he estado realizando mis prácticas. En ese momento, la empresa había comenzado a desarrollar este proyecto y me integré en él desde sus primeras fases. Desde entonces, hemos trabajado en su ampliación y mejora, logrando que la aplicación sea apta para su uso en distintas plantas de la empresa a nivel global.

Debido a mi implicación directa en este proyecto, decidí continuar trabajando en esta herramienta y enfocarla como tema de mi Proyecto de Fin de Ciclo.

2. ORGANIZACIÓN DE LA MEMORIA

A continuación, se presenta la estructura general del presente documento, organizada en distintos apartados que reflejan todas las fases del desarrollo del proyecto, desde la idea inicial hasta su implementación y evaluación final:

- 1. Identificación del Proyecto:** Expone el origen y la motivación del proyecto, así como el contexto en el que se desarrolla.
- 2. Identificación del Proyecto:** Organización de la Memoria: Describe cómo está estructurado el documento, indicando los contenidos de cada apartado.
- 3. Descripción General del Proyecto:** Introduce los objetivos del trabajo, la metodología utilizada durante su desarrollo y el entorno de trabajo, incluyendo las tecnologías y herramientas empleadas.
- 4. Descripción General del Producto:** Presenta una visión global del sistema desarrollado, sus funcionalidades básicas, su arquitectura y las tecnologías utilizadas para su despliegue.
- 5. Planificación y Presupuesto:** Detalla la planificación temporal del proyecto mediante cronogramas, así como una estimación de los recursos y costes asociados.
- 6. Documentación Técnica:** Constituye el núcleo del trabajo. Incluye los requisitos funcionales y no funcionales, el análisis del sistema, el diseño (base de datos, interfaz y lógica de aplicación), la implementación del código, así como las pruebas realizadas para validar el correcto funcionamiento del producto.

7. **Manuales de Usuario:** Incluye dos manuales diferenciados: uno destinado al usuario final para el uso de la aplicación y otro orientado a la instalación y puesta en marcha del sistema
8. **Conclusiones y Posibles Ampliaciones:** Recoge una reflexión final sobre el desarrollo del proyecto, los objetivos alcanzados, las dificultades encontradas y propuestas de mejora o ampliación futuras.
9. **Bibliografía:** Relación de fuentes consultadas durante el desarrollo del trabajo, tanto técnicas como conceptuales.
10. **Anexos:** Se incluyen documentos complementarios que apoyan el desarrollo del proyecto, como el diagrama de Gantt utilizado para la planificación

3. DESCRIPCIÓN GENERAL DEL PROYECTO

3.1. Objetivos

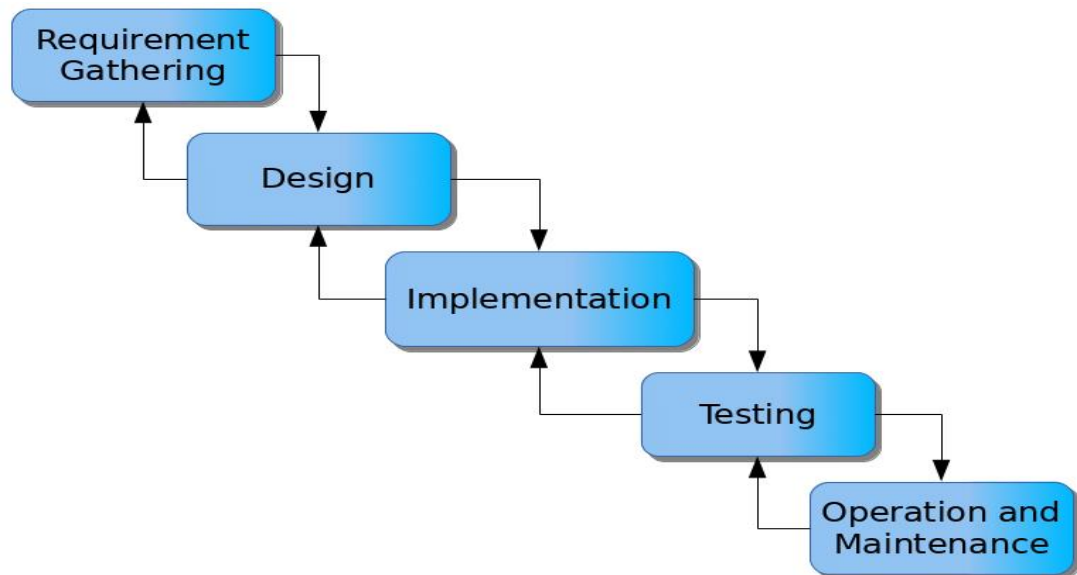
El principal objetivo que se pretende alcanzar con este proyecto es el desarrollo de una aplicación real, funcional y útil. Además, durante su realización, se persiguen otros objetivos que pueden agruparse en dos categorías: aquellos relacionados con la realización del proyecto y personales.

- Relacionados con el desarrollo del proyecto:
 - Diseñar una interfaz intuitiva y accesible para mejorar la experiencia del usuario.
 - Integrar una base de datos para el almacenamiento de la información.
 - Incluir una API para obtener datos en tiempo real.
 - Documentar el código y el proceso de desarrollo para facilitar futuras mejoras.
- Personales:
 - Aprender a utilizar nuevas tecnologías y frameworks aplicados en el proyecto.
 - Mejorar mis habilidades en desarrollo web y bases de datos.
 - Fortalecer mis habilidades de resolución de problemas y toma de decisiones.
 - Adquirir experiencia en la gestión y desarrollo de proyectos tecnológicos.

3.2. Cuestiones metodológicas

Desde mi perspectiva, las cuestiones metodológicas asociadas al proyecto están directamente relacionadas con el enfoque seguido para su desarrollo. Dado que se trata de un proyecto unipersonal y es necesario abordar cada una de las fases que conforman un proyecto tradicional, he optado por emplear el modelo de Ciclo de Vida en Cascada con Retroalimentación.

Este modelo de desarrollo me permite avanzar de forma secuencial a través de las distintas etapas del proyecto, con la posibilidad de retroceder a fases anteriores en caso de que sea necesario realizar ajustes o correcciones, lo cual resulta especialmente útil en un proceso iterativo como el que requiere este tipo de aplicación.



Ciclo de Vida en Cascada con Retroalimentación (fuente: [researchgate.net](https://www.researchgate.net))

3.3. Entorno de trabajo (tecnologías de desarrollo y herramientas)

El entorno de trabajo para el desarrollo del proyecto será el equipo de la empresa que he estado utilizando durante todo el desarrollo de mis prácticas:

- HP EliteBook 660 16 inch G11 Notebook PC, Intel® Core™ Ultra 5 135U y 1.60 GHz.
- Sistema operativo: Windows 10 Enterprise.

Respecto a las tecnologías y herramientas utilizadas:

- Base de datos: MySQL Community Server versión 8.
- Entorno de trabajo integrado: Visual Studio Code.
- Backend: Node.js y Express.js.
- Frontend: TailwindCSS y librerías de JavaScript.
- Control de Versiones (CVS): Git y GitHub.

4. DESCRIPCIÓN GENERAL DEL PRODUCTO

El producto final será un sistema desarrollado como una aplicación web, orientado a la gestión de la saturación de los distintos puestos de trabajo asignados a los carretilleros. Su objetivo principal es proporcionar una plataforma centralizada que permita organizar y administrar de forma eficiente la información relacionada con dichos puestos, así como con las etapas y actividades que se llevan a cabo en cada uno de ellos.

4.1. Visión general del sistema

El sistema desarrollado es una aplicación web diseñada para gestionar y optimizar la saturación de los distintos puestos de trabajo asignados a los carretilleros. Su objetivo principal es proporcionar una plataforma centralizada que permita organizar, administrar y analizar la información relacionada con los puestos, etapas y actividades realizadas en cada uno de ellos.

Límites del sistema

El sistema está diseñado para operar exclusivamente en un entorno web, accesible desde navegadores modernos y está centrado exclusivamente en la gestión de la información asociada a los **puestos de trabajo, etapas y actividades** realizadas por los carretilleros. No contempla la gestión completa de recursos humanos, inventarios u otras áreas externas al flujo operativo del puesto. Además, la funcionalidad está limitada a los datos que se encuentren registrados en la base de datos propia del sistema, sin integración directa (por ahora) con otros sistemas empresariales como ERP o MES.

Funcionalidades básicas

Entre las funcionalidades básicas se encuentra la creación y eliminación de puestos de trabajo, la gestión de etapas dentro de dichos puestos y la modificación de parámetros como la distancia o el número de picadas. También permite visualizar en detalle la información relacionada con cada etapa, como las operaciones, los métodos o la saturación del puesto. A nivel general, el sistema ofrece la posibilidad de consultar la saturación por puesto o por etapas, utilizando representaciones gráficas que facilitan su análisis.

Usuarios

El sistema está pensado principalmente para ser utilizado por los jefes de taller (JT) o responsables de logística, quienes tienen la responsabilidad de distribuir las tareas entre los distintos puestos de trabajo y evaluar el grado de saturación de cada uno. También podrían hacer uso del sistema técnicos de procesos o planificación, según las necesidades específicas de cada planta.

Sistemas con los que puede interactuar

En su versión actual, la aplicación funciona como un sistema parcialmente autónomo, ya que únicamente interactúa con una base de datos MySQL para almacenar la información y permitir consultas y actualizaciones en tiempo real, garantizando así la consistencia de los datos. No obstante, está diseñada para poder integrarse en el futuro con otros sistemas de la empresa, como bases de datos corporativas o herramientas de planificación, mediante API u otras interfaces de comunicación.

4.2. Descripción de métodos, técnicas o arquitecturas utilizadas

Para el desarrollo de la aplicación se ha utilizado una arquitectura cliente-servidor, en la que el frontend y el backend están claramente separados. Esta decisión permite un mayor modularidad, facilitando tanto el mantenimiento como la escalabilidad del sistema. El backend está construido utilizando Node.js junto al framework Express.js, lo que permite gestionar las peticiones de los usuarios de manera eficiente y estructurada.

Toda la información se almacena en una base de datos relacional MySQL. Esta base de datos permite mantener la integridad de los datos y realizar consultas complejas para obtener información precisa en tiempo real. A través de operaciones SQL bien definidas, el sistema es capaz de ofrecer datos actualizados al usuario, lo que resulta esencial para el análisis de saturación y la toma de decisiones en planta.

En la parte del cliente, se ha desarrollado una interfaz web utilizando tecnologías como HTML, JavaScript y TailwindCSS. Esta combinación proporciona una experiencia visual limpia y funcional, lo que facilita la interacción por parte de los usuarios. La interfaz permite realizar acciones como la creación y eliminación de puestos y etapas, la visualización gráfica de la saturación o la modificación de parámetros como la distancia o el número de picadas.

Como metodología de desarrollo se ha empleado el modelo en cascada con retroalimentación. Este enfoque ha permitido trabajar por fases ordenadas, desde el análisis inicial hasta la implementación y pruebas finales, pero dejando abierta la posibilidad de volver atrás cuando ha sido necesario introducir correcciones o mejoras. Esta flexibilidad ha sido especialmente útil al tratarse de un proyecto desarrollado de forma individual, donde la planificación y la revisión continua han sido clave para alcanzar un resultado funcional y estable.

4.3. Despliegue de la aplicación

El despliegue de la aplicación implica la configuración de la plataforma tecnológica, la instalación de los componentes necesarios y la puesta en marcha del sistema. A continuación, se describen los pasos y requisitos para llevar a cabo este proceso.

Plataforma tecnológica

La aplicación está desarrollada para ejecutarse en un entorno web, combinando distintas tecnologías tanto en el lado del cliente como en el servidor. En el servidor backend se ha utilizado Node.js como plataforma base, acompañado del framework Express.js, que facilita la gestión de rutas y peticiones HTTP. Toda la información del sistema se almacena y consulta a través de una base de datos MySQL, que permite una gestión eficiente y estructurada de los datos.

En cuanto al cliente frontend, se ha empleado HTML y TailwindCSS para definir la estructura y el diseño visual de la interfaz, junto con JavaScript para la lógica de interacción entre el usuario y el servidor. Además, el uso de EJS (Embedded JavaScript) permite generar vistas dinámicas en el cliente a partir de los datos procesados por el servidor.

El sistema puede ejecutarse en múltiples plataformas, siendo compatible con sistemas operativos como Windows, Linux o macOS. Asimismo, se accede a través de navegadores modernos como Google Chrome, Mozilla Firefox o Microsoft Edge. Para su despliegue en producción se recomienda utilizar servicios en la nube como AWS o Azure, aunque también es posible alojarlo en un servidor físico siempre que cuente con los requisitos necesarios para ejecutar Node.js y gestionar una base de datos MySQL.

Instalación de la aplicación

Para su instalación es necesario tener instalado Node.js (versión 14 o superior) y MySQL (versión 5.7 o superior), así como tener configurado un entorno de línea de comandos (como PowerShell, Bash o Terminal). A continuación, se detallan los pasos a seguir para una correcta y completa instalación de la aplicación:

1. Clonar el repositorio:

```
git clone https://github.com/carlafb-horse/APP_CARRETILLERO.git
cd APP_CARRETILLERO
```

2. Instalar las dependencias:

```
npm install
```

3. Configurar la base de datos:

- Crear una base de datos en MySQL con el nombre *carretillero*.
- Importar el archivo *database.sql* para crear las tablas necesarias:

```
mysql -u root -p carretillero < database.sql
```

- Configurar las credenciales de la base de datos en el código (index.js – línea 12):

```
//Creamos el pool de conexiones
const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'toor',
  database: 'carretillero',
  port: 3306
});
```

4. Iniciar el servidor:

```
npm start
```


Puesta en marcha

Tras realizar los pasos detallados anteriormente, la aplicación ya estará preparada para su uso. Para acceder a ella solamente hay que acceder, desde un navegador web a esta URL: <http://localhost:3000>. Una vez ahí, ya se podrán probar las funciones principales.

5. PLANIFICACIÓN Y PRESUPUESTO

En primer lugar, definiré las tareas que forman parte del proyecto asignando un tiempo estimado a cada una de ellas.

La siguiente tabla muestra el resumen del tiempo estimado:

FASES	100 HORAS
Análisis y diseño	15 horas
Configuración del entorno	10 horas
Desarrollo del proyecto	40 horas
Pruebas y depuración	20 horas
Documentación del proyecto	10 horas
Presentación y despliegue	5 horas

A continuación, se detalla la planificación de cada una de las fases:

FASE: Análisis y diseño	15 HORAS
Definición de requisitos	5 horas
Diseño de la base de datos	5 horas
Diseño de la arquitectura de software	5 horas

FASE: Configuración del entorno	10 HORAS
Insertión de datos en la base de datos	4 horas
Configuración del entorno de desarrollo	2 horas
Inicialización del proyecto backend	4 horas

FASE: Desarrollo del proyecto	40 HORAS
Módulo de gestión de puestos	20 horas
Módulo de gestión de etapas	20 horas

FASE: Pruebas y depuración	20 HORAS
Pruebas unitarias	4 horas
Pruebas de integración y de API	8 horas
Depuración de errores y optimización	8 horas

FASE: Documentación del proyecto		10 HORAS
Documentación técnica		6 horas
Documentación de usuario y manual de usuario		4 horas

FASE: Presentación y despliegue		5 HORAS
Preparación de la presentación		6 horas
Despliegue de la aplicación		4 horas

6. DOCUMENTACIÓN TÉCNICA

6.1. Especificación de requisitos

La aplicación debe permitir al usuario gestionar los **puestos de trabajo**, ofreciendo la posibilidad de añadir nuevos puestos especificando su número, nombre, cantidad de operarios asignados, turno y planta. También debe facilitar la eliminación de puestos existentes, así como mostrar una lista completa de todos los puestos registrados. Al agregar un nuevo puesto con cualquier número definido por el usuario, este se insertará con el número indicado, desplazando al resto de puestos para mantener el orden.

En cuanto a las **etapas** operativas, el sistema debe permitir registrar nuevas etapas vinculadas a un puesto específico, con la opción de modificar ciertos datos como el número de picadas, las distancias recorridas y los tiempos de actividad. Además, debe permitir eliminar etapas y mostrar una lista de las etapas asociadas a cada puesto.

Respecto a las **referencias**, la aplicación debe ofrecer funcionalidades para buscar y validar referencias específicas, vincularlas a las etapas correspondientes y mostrar listados de referencias asociadas a cada etapa.

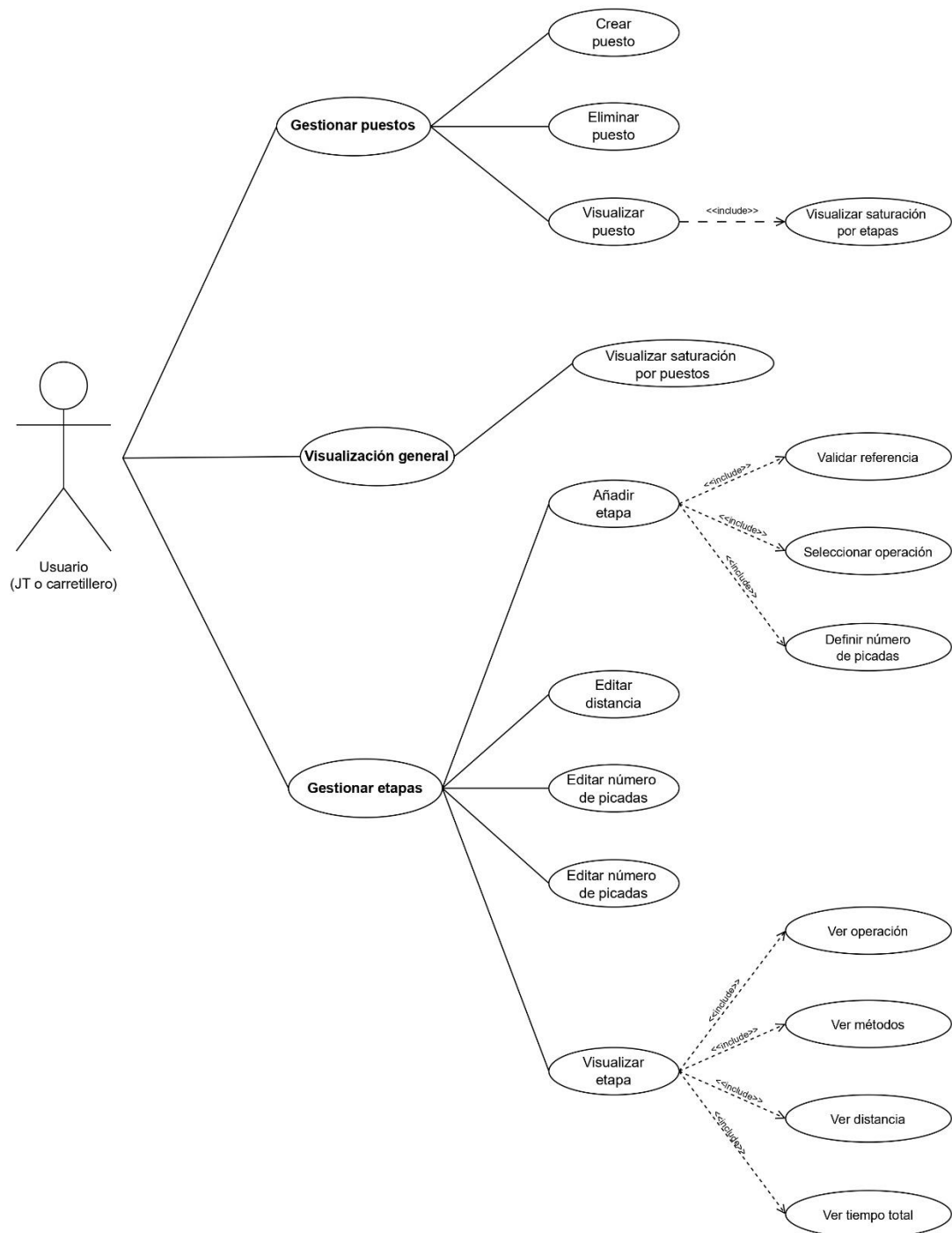
La **visualización de datos** es fundamental, por lo que el sistema debe generar gráficos interactivos que ayuden a analizar la saturación de los puestos y el rendimiento de las actividades realizadas. El usuario debe poder elegir el tipo de gráfico a visualizar y estos deben actualizarse en tiempo real conforme se registren nuevos datos.

El sistema debe permitir el **registro manual de datos** como distancias recorridas y cantidades manipuladas, y ofrecer una consulta detallada de la información organizada por puesto y etapa.

Finalmente, la aplicación debe ser **accesible** desde cualquier navegador web moderno, con una interfaz intuitiva garantizando así una experiencia de usuario óptima.

6.2. Análisis del sistema

Para un mejor análisis del sistema, se muestra el diagrama de casos de uso de la aplicación para así comprender mejor las funcionalidades del propio sistema.



A continuación, se detallan cada una de las funcionalidades principales representadas en el diagrama de casos de uso. Este análisis tiene como objetivo descomponer y describir con mayor precisión cómo interactúa el usuario con el sistema, qué operaciones puede realizar y qué lógica hay detrás de cada acción. De esta forma, se proporciona una visión más profunda de las capacidades del sistema.

Visualización general

- **Visualizar saturación por puesto:** permite visualizar los puestos y su saturación.

El sistema carga todos los puestos junto con su saturación correspondiente y la dispone en un gráfico de barras acompañado de la media de saturación de los puestos. Cada puesto tiene un color dependiendo de su turno.

Gestión de puestos

- **Crear puesto:** permite registrar un nuevo puesto de trabajo en el sistema.

El usuario debe introducir los datos necesarios para crear un puesto, número y nombre del puesto, número de operarios asignados y seleccionar el turno de un desplegable. El sistema valida que los datos tengan el formato correcto y comprueba que todos los campos obligatorios estén cumplimentados. Una vez superadas estas validaciones, el puesto se inserta en la base de datos y, tras una recarga automática de la pantalla, aparece reflejado junto al resto de puestos.

- **Eliminar puesto:** permite eliminar un puesto de trabajo que ya no sea necesario.

El usuario selecciona un puesto y pulsa el botón de eliminar, el sistema lo elimina de la base de datos y este desaparece de la lista.

- **Visualizar puesto:** permite visualizar la saturación de un puesto de trabajo.

El sistema obtiene todos los puestos, junto con su saturación, de la base de datos, de forma que el usuario los visualiza en la pantalla principal de la aplicación. Cuando selecciona un puesto, se obtiene la saturación desglosada por etapas del mismo.

Gestión de etapas

- **Crear etapa:** permite registrar nuevas etapas asociadas a un puesto específico.

El usuario debe seleccionar un puesto y pulsar sobre el botón de añadir una nueva etapa, después debe introducir los datos requeridos, la referencia, el número de picadas (embalajes a mover en simultáneo) y seleccionar la operación que realizarán esas referencias. Las referencias también se pueden seleccionar a través del buscador, en el que se dispondrán todas las referencias disponibles para ese puesto en ese turno.

- **Editar distancia:** permite modificar la distancia recorrida en una etapa específica.

El usuario selecciona una etapa de un puesto y pulsa sobre el icono del plano blanco, introduce el nuevo valor de la distancia y al pulsar sobre el botón de actualizar, el sistema actualiza la distancia de todas las etapas de referencias y se actualiza el tiempo total y el tiempo total por picadas.

- Editar número de picadas: permite modificar las picadas en una etapa específica.

El usuario selecciona una etapa de un puesto y pulsa sobre el icono de información gris, selecciona el nuevo número de picadas y al pulsar sobre el botón de actualizar, el sistema actualiza el número de picadas de todas las etapas de referencias y se actualiza el tiempo total por picadas.

- Visualizar etapa: permite visualizar todos los datos de una etapa específica.

El usuario selecciona una etapa global de un puesto, tras lo cual se desplegarán las etapas correspondientes a las distintas referencias. Al pulsar sobre cualquiera de ellas, se muestra una tabla con toda la información relacionada con esa etapa de referencia.

En esta tabla se visualizan datos como los métodos asociados a la etapa global, incluyendo su símbolo y el tiempo necesario para su ejecución, el número de embalajes a mover y los minutos totales empleados. También se muestran la distancia total recorrida, la velocidad estimada y el tiempo total invertido en el desplazamiento. Asimismo, se presenta la actividad total en minutos, la actividad por picada y el número de picadas requeridas.

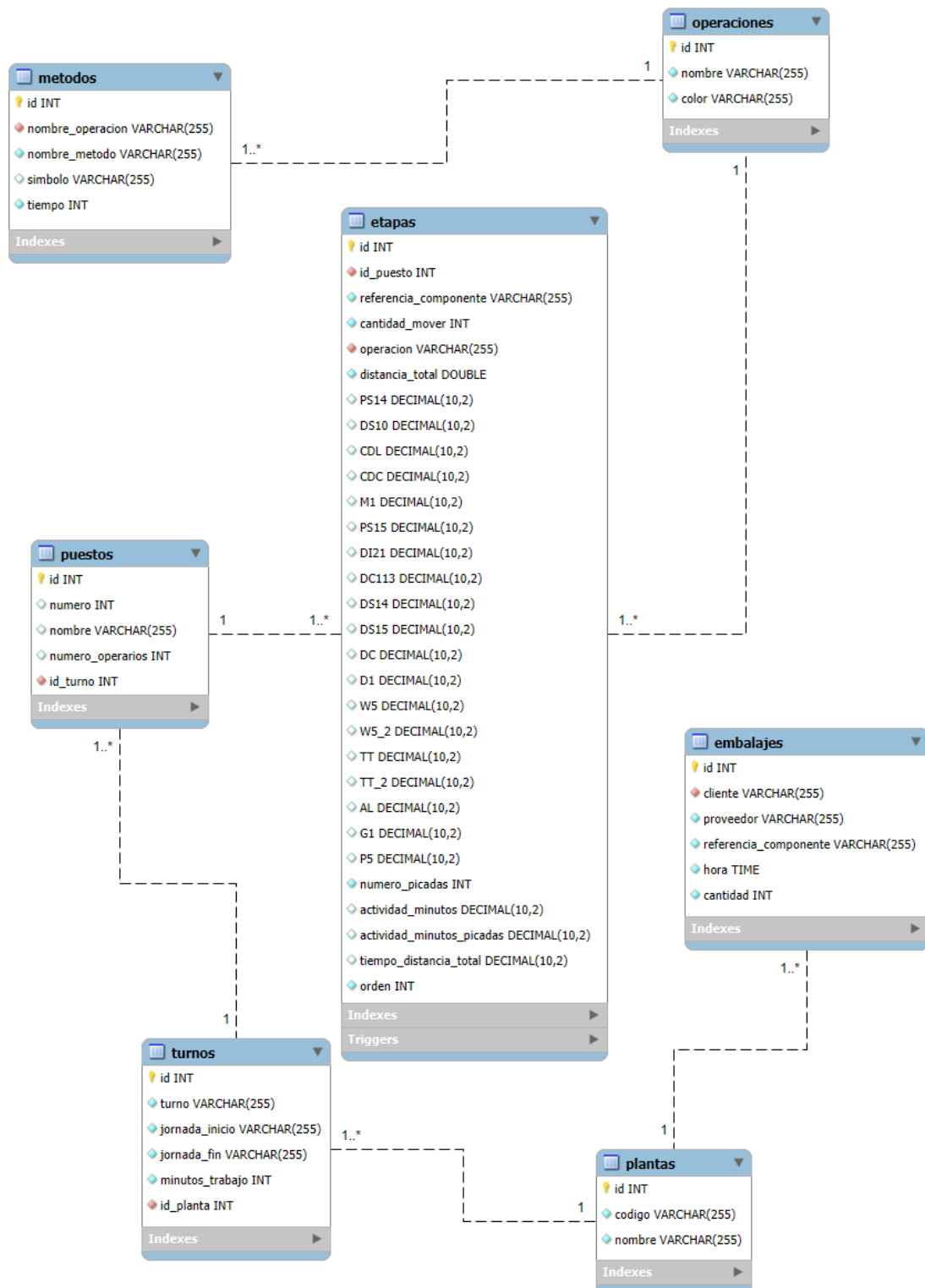
Cada etapa global pertenece a una operación, y está representada por un color distintivo, el cual coincide con el color utilizado en el gráfico de saturación correspondiente al puesto.

6.3. Diseño del sistema

6.3.1. Diseño de la Base de Datos

A continuación, se muestra el diagrama entidad-relación de la base de datos empleada para el desarrollo de la aplicación. Este diagrama permite visualizar de forma estructurada las diferentes entidades que conforman el sistema, así como las relaciones que existen entre ellas.

En él se representan las tablas principales, los atributos de cada entidad y las relaciones entre ellas.



- Tabla 'plantas':

Esta tabla recoge información relativa a las plantas, como el código identificador y el nombre de cada una. Estos datos son esenciales para consultar información asociada, como los turnos o los puestos correspondientes a cada planta.

- Tabla ‘turnos’:

En esta tabla se incluyen los datos correspondientes a los turnos de trabajo. Entre los campos más relevantes se encuentran la hora de inicio y de fin de cada turno, estos valores serán utilizados para filtrar la información que se va a procesar.

- Tabla ‘puestos’:

Esta tabla almacena información sobre los distintos puestos de trabajo, incluyendo datos como el número de puesto y el turno al que pertenecen. Estos campos son especialmente relevantes, ya que permiten organizar y visualizar los puestos en función del turno correspondiente.

- Tabla ‘metodos’:

Esta tabla contiene información detallada sobre los métodos que conforman las distintas operaciones. Entre los datos registrados se incluyen, entre otros, el símbolo asociado a cada método y su tiempo de ejecución. Esta información resulta fundamental para el cálculo de la saturación de los puestos de trabajo, y proporciona datos relevantes que facilitan la comprensión al usuario.

- Tabla ‘operaciones’:

Esta tabla almacena información relativa a las operaciones, incluyendo el nombre y el color asociado a cada una de ellas. Estos datos serán utilizados en la visualización de las etapas, facilitando su identificación y comprensión por parte del usuario.

- Tabla ‘embalajes’:

La tabla proporciona información relacionada con los embalajes, incluyendo la cantidad necesaria para transportar cada referencia y la hora de recepción de estos. Estos datos son fundamentales para calcular el tiempo que se invierte en la recepción de cada referencia, lo cual influye directamente en el cálculo de la saturación de los puestos de trabajo.

- Tabla ‘etapas’:

Esta es una de las tablas clave del sistema, ya que en ella se almacenan los datos correspondientes a las distintas etapas del proceso, a partir de los cuales se calcula la saturación de los puestos de trabajo. Entre los campos más relevantes se encuentra la cantidad a mover, que hace referencia a la columna cantidad de la tabla embalajes, así como la distancia total, un valor esencial para el cálculo de la saturación. También se incluye el número de picadas, que indica cuántos embalajes transportará simultáneamente el operario, el orden, que permite organizar las etapas de manera secuencial, y la actividad en minutos por picada, que refleja directamente el nivel de saturación del puesto. La combinación de estos valores permite realizar un análisis detallado de la carga de trabajo y optimizar la distribución de tareas en el entorno de producción.

6.3.2. Diseño de la Interfaz de usuario

La interfaz de usuario de la aplicación ha sido diseñada con un enfoque centrado en la funcionalidad, claridad visual y facilidad de uso. Cada sección está pensada para guiar al usuario a través de las distintas funcionalidades del sistema sin generar confusión, presentando la información de forma ordenada y accesible.

Desde el primer acceso a la aplicación, se presenta una vista general que permite al usuario tener una panorámica de los puestos de trabajo existentes y su nivel de saturación. Esta vista principal actúa como panel de control desde el cual se puede acceder a la información de cada puesto.

A continuación, se muestra una captura de la pantalla principal de la aplicación:



La pantalla principal está compuesta por una barra superior que incluye el título de la aplicación, una barra lateral con un panel de opciones e información, y un panel central donde se visualizan un gráfico de barras con todos los puestos registrados, con un color representativo del turno de trabajo y una línea que representa la media de saturación de los puestos. Desde esta pantalla el usuario puede tanto crear nuevos puestos, como acceder a información más detallada de cada uno de ellos.

La siguiente imagen muestra el formulario de creación de un nuevo puesto:

Al pulsar el botón “Añadir puesto”, se despliega un modal donde el usuario debe introducir los datos requeridos: número del puesto, nombre, número de operarios, y seleccionar el turno. Este formulario incluye validaciones para evitar registros incompletos o con datos incorrectos. Una vez completado, el nuevo puesto se incorpora automáticamente al panel principal.

Panel de opciones

Valladolid

Legenda colores distancia

- De 0 a 49 metros
- De 50 a 99 metros
- 100 o más metros

Jornada laboral: Mañana ☀️

Inicio: 07:00:00 Fin: 15:00:00

Añadir puesto

Añadir etapa

APP CARRETERILERO

Puesto: Almacenero

Actividad	Porcentaje (%)
Almacenamiento	~85%
Des carga caja terminada	~10%
Gran empaque	~5%

Tareas

Tarea	Porcentaje (%)
Mañana	~90%
Tarde	~85%
Noche	~88%
Media	86.22%

Etapas disponibles

Eta	Descripción	Distancia (m)	Tiempo (min)	Iconos
Etapa: 5.	De estanteria a puesto inferior	101	268.04	[Info] [List] [Add] [Remove]
Etapa: 6.	Gestión de residuos	0	92.95	[Info] [List] [Add] [Remove]
Etapa: 8.	Zipado	0	45.63	[Info] [List] [Add] [Remove]

pág. 17

La siguiente imagen muestra la vista detallada de una etapa operativa:

Panel de opciones

Valladolid

Legenda colores distancia

- De 0 a 49 metros
- De 50 a 99 metros
- 100 o más metros

Jornada laboral: Mañana

Inicio: 07:00:00 Fin: 15:00:00

Añadir puesto Añadir etapa

Etapa: 5. De estantería a puesto inferior Distancia (m): 101 Tiempo (min): 268.04

Etapa: 5. De estantería a puesto inferior Componente: T853014271 1.59

Etapa: 5. De estantería a puesto inferior Componente: 297A31804R 25.38

Etapa: 5. De estantería a puesto inferior Componente: 7700112186 85.64

Operación	Símbolo (estándar MTM3)	Tiempo MTM3 (CTMin)		
TOMA SIMPLE en estantería de embalaje lleno (altura media)	PS15	44	54	23.76
DEPOSITA en instalación	DI21	30	54	16.20
CAMBIO de dirección libre al llegar a estantería	CDL	6	54	3.24
Distancia	Metros	Velocidad	64	32.72
	101	0.6		
Actividad total en minutos				85.64
Actividad en minutos (según el número de picadas simultáneas)				85.64
Número de picadas				1

Etapa: 5. De estantería a puesto inferior Componente: R100577958 1.59

Etapa: 5. De estantería a puesto inferior Componente: 323200520R 15.86

Etapa: 5. De estantería a puesto inferior Componente: 304011184R 38.06

Etapa: 5. De estantería a puesto inferior Componente: 323723548R 3.17

Etapa: 5. De estantería a puesto inferior Componente: 322754172R 41.24

Etapa: 5. De estantería a puesto inferior Componente: 328010161R 4.76

En esta vista, se muestra una tabla completa con los datos técnicos de la etapa: métodos involucrados con su símbolo y duración, cantidad de embalajes manipulados, distancia recorrida, velocidad media, tiempos empleados y número de picadas. Esta información es clave para el análisis y la mejora del rendimiento en cada puesto.

La siguiente imagen muestra el proceso de añadir una nueva etapa a un puesto:

Rellene los campos para añadir una etapa

Referencia del componente:

Ingresa la referencia del componente

Número de embalajes a mover en simultáneo: Operación:

1 1. Descarga camión en muelle

Añadir etapa

Para poder registrar una etapa, el usuario debe haber seleccionado previamente un puesto de trabajo, ya que todas las etapas deben estar asociadas a uno en concreto. Al pulsar el botón de “Añadir etapa”, se despliega un formulario que permite registrar los datos necesarios para definir la operación.

En este formulario, el usuario debe introducir las referencias que desee añadir a la etapa o buscarlas y seleccionarlás. A continuación, el usuario debe completar los campos específicos de la etapa, incluyendo el número de picadas y la operación a realizar.

Una vez introducida toda la información, se valida su formato y se registra la etapa en la base de datos. Al finalizar el proceso, la aplicación recarga de forma automática la vista de etapas, mostrando la nueva etapa junto a las existentes, codificada con su color correspondiente en el gráfico de saturación. Esto permite mantener una visión clara y actualizada del estado operativo del puesto.

6.3.3. Diseño de la Aplicación

La aplicación ha sido diseñada para ofrecer una estructura clara y funcional que permita una gestión eficiente de los puestos de trabajo, etapas y operaciones logísticas relacionadas con el trabajo de los carretilleros. A continuación, se enumeran y detallan los procesos reales implementados en la aplicación durante su desarrollo:

Carga inicial del sistema

Al iniciar la aplicación se realiza una consulta a la base de datos para obtener todos los puestos registrados. Se renderiza el gráfico de saturación por puesto, mostrando los colores asignados a cada etapa.

Creación de puestos

El usuario accede al formulario de creación mediante el botón “Añadir puesto”. Se introducen el número y nombre del puesto, el número de operarios, el turno y la planta. El sistema valida que todos los campos estén correctamente rellenos y con el formato adecuado. Tras insertar el puesto en la base de datos, se recarga la página automáticamente y se actualiza el gráfico, desplazando los puestos si es necesario para insertar el nuevo.

Eliminación de puestos

El usuario puede eliminar un puesto específico desde su tarjeta visual. Al eliminarlo, se actualiza automáticamente el gráfico y se reorganizan los puestos.

Creación de etapas

Solo es posible añadir etapas una vez seleccionado un puesto concreto. Al pulsar “Añadir etapa” se despliega un formulario dinámico para registrar la operación. Se introducen campos como las referencias, que también se pueden buscar, número de picadas y operación a realizar. La etapa queda vinculada al puesto y a la referencia correspondiente.

Edición de etapas

Desde la tabla de una etapa se puede editar el número de picadas o la distancia total, cada uno con un botón independiente. El sistema actualiza en tiempo real los cálculos y el gráfico de saturación.

Visualización de etapas

Desde cada puesto se despliegan sus etapas globales. Al seleccionar una etapa global, se muestran sus etapas de referencia en una tabla detallada. Se visualizan datos como el método y símbolo, tiempo de ejecución, número de embalajes, distancia total, velocidad, minutos totales y minutos totales por picada. El color de la etapa es el mismo que el que aparece en el gráfico de saturación por etapas, facilitando la identificación.

Eliminación de etapas

Es posible eliminar etapas de referencia desde su vista detallada. La eliminación implica una actualización automática del gráfico del puesto correspondiente.

Consulta y validación de referencias

El sistema permite buscar referencias antes de registrar una etapa filtrándolas por el turno con la hora de recepción de las mismas.

Visualización de gráficos

El gráfico principal refleja en todo momento el estado de saturación de los puestos, y el color de los puestos es representativo de su turno. A su vez, en el gráfico de saturación por etapa, cada etapa tiene un color identificativo que se corresponde con el gráfico. El gráfico se actualiza automáticamente al añadir, editar o eliminar etapas o puestos.

Gestión dinámica de modales

Los formularios para añadir y editar información se presentan en modales dinámicos. Se adaptan automáticamente a la operación en curso, optimizando el flujo del trabajo.

Actualización en tiempo real

Toda acción realizada, como crear, editar o eliminar, conlleva una actualización inmediata de la vista actual. No es necesario recargar manualmente la página para ver los cambios reflejados.

Este diseño secuencial y estructurado responde fielmente a la lógica seguida durante el desarrollo de la aplicación, garantizando una experiencia coherente, rápida y funcional para el usuario.

6.4. Implementación

6.4.1. Entorno de desarrollo

Durante el desarrollo de la aplicación se han utilizado herramientas específicas que han permitido organizar y construir de forma eficiente tanto el backend como el frontend, así como gestionar correctamente la base de datos. A continuación, se detalla el entorno de desarrollo empleado en el proyecto.

MySQL Workbench

Se ha utilizado como herramienta visual para la administración de la base de datos. Ha permitido realizar la comprobación de sentencias DML, así como diseñar y consultar el modelo Entidad/Relación de manera gráfica, facilitando la estructuración lógica de los datos almacenados.

Visual Studio Code

Ha sido el entorno de desarrollo principal, empleado para programar tanto la parte del servidor como la interfaz de usuario. Este editor ha permitido mantener una estructura organizada del proyecto, separar adecuadamente los archivos por funcionalidades y disponer de herramientas integradas para la depuración del código.

Para mejorar la experiencia de desarrollo y facilitar determinadas tareas, se han incorporado varias extensiones dentro de Visual Studio Code, como por ejemplo:

- Prettier: para el formateo automático del código y mantener una sintaxis limpia y homogénea.
- Chart.js: para facilitar el uso de la librería Chart.js, utilizada para la creación de gráficos interactivos en la aplicación.
- MySQL: para conectarse y consultar directamente la base de datos desde el propio entorno de desarrollo.
- Tailwind CSS IntelliSense: para autocompletado de clases de estilo dentro de los archivos del frontend.

Este conjunto de herramientas ha permitido una implementación ágil, controlada y estructurada, adaptada a las necesidades de una aplicación web con base de datos y gráficos interactivos.

6.4.2. Estructura del código

El código fuente de la aplicación está organizado siguiendo una estructura clara y modular, que facilita tanto el mantenimiento como la ampliación del sistema. La aplicación está dividida principalmente en dos partes: el backend (servidor) y el frontend (interfaz de usuario), cada una con sus propias carpetas y archivos.

El backend está implementado en Node.js utilizando el framework Express.js, y sigue un enfoque estructurado-procedimental, aunque en ciertos archivos se emplea también un estilo orientado a eventos, especialmente en la gestión de solicitudes HTTP y en la conexión con la base de datos. El código está dividido en archivos según su funcionalidad, separando rutas, controladores y lógica de conexión con la base de datos. Se utiliza el sistema de módulos de Node.js para importar y exportar funciones de forma ordenada.

Por otro lado, el frontend está compuesto por archivos EJS (Embedded JavaScript) para la generación dinámica de vistas, y utiliza HTML, TailwindCSS y JavaScript como tecnologías principales. El comportamiento de la interfaz se gestiona mediante funciones estructuradas y eventos del DOM, lo que da lugar a un enfoque orientado a eventos. Este modelo facilita la interacción del usuario con elementos como botones, formularios y gráficos.

Además de las librerías nativas de Node.js, se han descargado e instalado las siguientes dependencias externas a través de npm:

- express: para la gestión del servidor y enrutamiento.
- mysql: para la conexión con la base de datos MySQL.
- dotenv: para la gestión de variables de entorno.
- ejs: para el renderizado de plantillas dinámicas en el frontend.
- body-parser: para procesar los datos recibidos en las solicitudes POST.
- chart.js: para la visualización de gráficos interactivos en el navegador.

La organización del código permite un flujo lógico desde la entrada del usuario hasta el procesamiento en el servidor y la respuesta en la interfaz, manteniendo una separación clara entre lógica de negocio, presentación y acceso a datos.

A continuación, se analiza en detalle el código de los archivos más relevantes de la aplicación, con el objetivo de comprender su estructura y funcionalidad principal.

Fichero 'app.js'

Este es el núcleo del frontend de la aplicación. Contiene la lógica necesaria para gestionar la interacción del usuario con la interfaz, así como la comunicación con el backend mediante solicitudes HTTP. Este archivo es responsable de renderizar los datos en la interfaz, manejar eventos del usuario y actualizar dinámicamente los gráficos y tablas que representan la información de los puestos y etapas.

- Variables globales: se definen variables globales para almacenar datos importantes, como referencias de componentes, operaciones seleccionadas, y datos para gráfico.

```
//Creamos las variables globales que hacen falta
let referencia_componente = "", operacion_seleccionada, puestoID, conteosPorPuesto = [],
numero_picadas;

//Variable global donde almacenaremos en un diccionario la referencia y el número de embalajes
let referencia_embalaje = {};

//Variable que contiene los datos para el gráfico de chimenea
let conteoGraficoChimenea = [];

//Variable de control para gráfico del puesto
let chartPuesto = null;
```

- Gestión de puestos: obtiene los datos del backend, renderiza los puestos en gráficos y permite eliminar puestos (la agregación de nuevos puestos se realiza desde un archivo diferente: 'app.ejs').

```
async function fetchData() {
  // Obtenemos los PUESTOS
  try {
    //Almacenamos en una variable la respuesta de la llamada al end point para obtener los puestos
    const response = await fetch('/app/api/obtenerPuestos');

    //Controlamos la respuesta
    if (!response.ok) {
      throw new Error('Error fetching data');
    }

    //Almacenamos los puestos obtenidos
    const data = await response.json();

    //Llamamos al método para disponer los puestos en el panel superior
    gestionarPuesto(data);

    //Llamamos al método para obtener los datos para el gráfico de chimenea
    gestionarGraficoChimenea(data);
  } catch (exception) {
    console.error("Error al obtener los puestos: ", exception);
  }
}
```

Esta función obtiene los datos de los puestos desde el backend y llama a las funciones que gestionan los gráficos. La función 'gestionarPuestos()', organiza los datos de los puestos, los prepara para ser renderizados en gráficos y verifica si todas las peticiones necesarias han finalizado antes de renderizar los gráficos.

```
function eliminarRegistro(tipo, id_elemento, tabla, id_puesto) {
  //Preparamos la solicitud DELETE
  fetch(`/app/api/eliminarRegistro/${id_elemento}/${tabla}/${id_puesto}`, {
    method: "DELETE"
  })
  //Controlamos la respuesta
  .then(response => {
    //En caso de que todo haya salido bien
    if (response.status === 201) {
      switch (tipo) {
        case "puesto":
          mostrarAlerta("Eliminando puesto", null, null, 1);
          break;
        case "etapa_global":
          mostrarAlerta("Eliminando etapas", null, null, 1);
          break;
        case "etapa_referencia":
          mostrarAlerta("Eliminando etapa", null, null, 1);
          break;
      }
    }
    //En caso de que falle
  } else if (response.status === 501) {
    mostrarAlerta("Error", "Se ha producido un error", "error", 0);
  }
  //En casos no controlados
  } else {
    mostrarAlerta("Estado no controlado", "No se ha sido capaz de controlar el estado", "question", null);
  }
});
}
```

La eliminación de un puesto se gestiona mediante una función reutilizable que también se encarga de borrar etapas, lo que permite optimizar el código al centralizar la lógica de eliminación. Esta función envía una solicitud al backend para ejecutar la operación correspondiente y, además, muestra una alerta informativa al usuario para indicar el estado del proceso.

- Gestión de etapas: obtiene las etapas asociadas a un puesto y las organiza en tablas dinámicas, permite añadir nuevas etapas, editar valores como distancia o número de picadas, y eliminar etapas existentes.

```
function obtenerEtapas(puestoID) {
  //Iniciamos la solicitud GET para obtener las etapas de un puesto
  fetch(`/app/api/obtenerEtapasAgrupadasPuesto/${puestoID}`, {
    method: "GET"
  })
  //Controlamos la respuesta
  .then(response => {
    //En caso de que se produzca un error
    if (!response.ok) {
      throw new Error('Error fetching data');
    }

    //Devolvemos la información formateada
    return response.json();
  })

  //Controlamos los datos
  .then(data => {
    generarEtapaGlobal(data);
  });
}
```

Para obtenerlas etapas asociadas a un puesto, se hace una solicitud al backend y llama a la función que va a organizar esas etapas en una tabla.

```
function editarEtapa(id_puesto, operacion, num_picadas) {
  if (id_puesto === '' || id_puesto === null) {
    //Llamamos al método para mostrar una alerta de aviso
    mostrarAlerta('Error al mostrar el modal de añadir etapa', 'Debes de seleccionar un puesto antes de editar una etapa', 'error', 0);
  }
  //En cualquier otro caso...
} else {
  //Configuramos el modal
  [...]

  //Si se pasa el número de picadas como argumento
  if (num_picadas) {
    //Preseleccionamos el número de picadas en el dropdown
    const picadasSelect = document.getElementById('numeroPicadas');
    picadasSelect.value = num_picadas; // Establecemos el valor del número de picadas seleccionado
  }

  $('#actualizarEtapa').on('submit', async function (e) {
    //Paramos la propagación
    e.preventDefault();

    //Almacenamos en una variable el número de picadas
    numero_picadas = document.getElementById('numeroPicadas').value;

    //Preparamos la petición GET para actualizar la etapa
    fetch(`/app/api/actualizarEtapa/${id_puesto}/${operacion}/${numero_picadas}/1`, {
      method: "PUT"
    })
    // Controlamos los datos
    .then(response => {
      if (response.status === 200) {
        mostrarAlerta('Etapas actualizadas correctamente', null, null, 1);
      } else {
        mostrarAlerta('Error', 'Ha fallado', 'error', 0);
      }
    })
    .catch(error => {
      console.error('Error en la solicitud:', error);
      mostrarAlerta('Error', 'No se pudo conectar al servidor', 'error', 0);
    });
  });
}
```

Para editar el número de picadas de una etapa, se muestra un modal en el que está seleccionado por defecto el número de picadas actuales de esa etapa.

```
function editarDistancia(id_puesto, operacion) {
  //Misma estructura que en la edición del número de picadas
  [...]
}
```


Para editar la distancia de una etapa seleccionada, aparece un modal en el que se debe introducir la nueva distancia.

```
function generarEtapaGlobal(etapas) {
  let contenedorTablas = document.getElementById('tablaEtapasGlobal');
  contenedorTablas.innerHTML = '';

  etapas.forEach((etapa) => {
    //Declaramos una variable para almacenar el color de la etapa
    const color_etapa = etapa.color;

    let color_distancia;

    //Creamos una variable para almacenar el nombre de la etapa
    const f = etapa.name

    var { nombre_etapa, id_etapa, id_puesto, distancia_total, actividad_minutos_picadas, num_picadas } = inicializarVariablesEtapas(etapa);

    //Creamos un if para controlar la distancia total de la etapa y así poder modificar el color de la misma.
    //En caso de la distancia sea de 0 a 49
    if (distancia_total >= 0 && distancia_total <= 49) {
      //Asignamos el color verde
      color_distancia = 'text-green-700';

      //En caso de que la distancia sea entre de 50 a 100
    } else if (distancia_total >= 50 && distancia_total <= 99) {
      //Asignamos el color amarillo
      color_distancia = 'text-yellow-700';

      //En caso de que la distancia sea más de 100
    } else if (distancia_total >= 100) {
      //Asignamos el color rojo
      color_distancia = 'text-red-700';

      //En cualquier otro caso
    } else {
      //Asignamos el color azul
      color_distancia = 'text-blue-700';
    }

    //Asignamos el puesto
    id_puesto = etapa.id_puesto;

    //Generamos el HTML de la tabla para la etapa
    const tablaHTML = `
    <div id="etapa-${id_puesto}-${nombre_etapa}" class="mb-4" data-id-etapa="${id_etapa}">
      <h3 id="encabezadoEtapa-${nombre_etapa}"
        class="text-lg font-semibold mb-2 p-2 rounded-lg animate-fadeIn text-black ${f === 'X' ? 'bg-stone-200' : ''}"
        style="${f !== 'X' ? 'background-color: ${color_etapa};' : ''}">

        <div class="grid grid-cols-10 gap-4 w-full">
          <span class="col-span-4" onclick="toggleEtapas(${id_puesto}, '${nombre_etapa}')">
            Etapa: <strong>${nombre_etapa}</strong></span>
          <span class="col-span-2" onclick="toggleEtapas(${id_puesto}, '${nombre_etapa}')">
            Distancia (m): <strong class="${color_distancia}">${distancia_total}</strong></span>
          <span class="col-span-2" onclick="toggleEtapas(${id_puesto}, '${nombre_etapa}')">
            Tiempo (min): <strong>${actividad_minutos_picadas.toFixed(2)}</strong></span>

          <div class="col-span-2 flex justify-between gap-2">
            <button type="button" class="text-gray-500"
              onclick="editarEtapa(${id_puesto}, '${nombre_etapa}', '${num_picadas}')">
              <i class="bi bi-info-circle-fill" style="font-size: 20px;"></i>
            </button>

            <button type="button" class="text-white"
              onclick="editarDistancia('${id_puesto}', '${nombre_etapa}')">
              <i class="bi bi-map"></i>
            </button>

            <button type="button" class="text-blue-500"
              onclick="anyadirEtapa(${id_puesto}, '${nombre_etapa}', '${num_picadas}')">
              <i class="bi bi-plus-circle-fill" style="font-size: 20px;"></i>
            </button>

            <button type="button" class="text-red-500"
              onclick="eliminarRegistro('etapa_global', '${nombre_etapa}', 'etapas', ${id_puesto})">
              <i class="bi bi-trash-fill"></i>
            </button>
          </div>
        </div>
      </h3>
    </div>
  `;
    contenedorTablas.insertAdjacentHTML('beforeend', tablaHTML);
  })
}
```

Para generar las etapas globales, se renderizan las etapas asociadas a un puesto en tablas dinámicas que muestran toda la información relevante de cada una, como su nombre, la distancia recorrida — representada con diferentes colores según el tipo de desplazamiento — y el tiempo total por picada empleado. Además, cada tabla incorpora botones que permiten gestionar las etapas fácilmente: editar el número de picadas, modificar la distancia, añadir etapas con las mismas características o eliminarlas del sistema.

```
function anyadirEtapa(id_puesto, operacion, num_picadas) {
  let opcion;
  //Controlamos el valor del argumento "id"... en caso de que el argumento sea NULL
  if (id_puesto === '' || id_puesto === null) {
    //Llamamos al método para mostrar una alerta de aviso
    mostrarAlerta('Error al mostrar el modal de añadir etapa', 'Debes de seleccionar un puesto antes de añadir una etapa', 'error', 0);
  }
  //En cualquier otro caso...
  else {
    //Configuramos el modal
    [...]

    //En caso de que la variable ya tenga referencias...
    if (referencia_componente !== null) {
      //Las añadimos al campo de las referencias
      document.getElementById('referencia_componente').value = referencia_componente;
    }

    // Si se pasa la operación como argumento
    if (operacion) {
      // Preseleccionamos la operación en el dropdown
      const operacionSelect = document.getElementById('operacion');
      operacionSelect.value = operacion; // Establecemos el valor de la operación seleccionada

      // Deshabilitamos el dropdown para evitar que el usuario lo cambie
      operacionSelect.disabled = true;
      opcion = 2;
    }

    // Si se pasa el número de picadas como argumento
    if (num_picadas) {
      // Preseleccionamos el número de picadas en el dropdown
      const picadasSelect = document.getElementById('numeroPicadas');
      picadasSelect.value = num_picadas; // Establecemos el valor del número de picadas seleccionado
      opcion = 2;
    }

    //Añadimos la funcionalidad para el formulario de consultar la información usando la referencia del componente y la línea
    $('#consultarValores').on('submit', async function (e) {
      //Paramos la propagación
      e.preventDefault();

      //Almacenamos en una variable global la referencia del componente
      referencia_componente = document.getElementById('referencia_componente').value;

      //Almacenamos en una variable el número de picadas
      numero_picadas = document.getElementById('numeroPicadas').value;

      //Almacenamos en la variable global la operación seleccionada
      operacion_seleccionada = document.getElementById('operacion').value;

      //Preparamos la petición GET para obtener las referencias válidas
      fetch('/app/api/comprobarReferencias/${referencia_componente}', {
        method: "GET"
      })
      //Controlamos la respuesta
      .then(response => {
        //En caso de que no este bien
        if (!response.ok) {
          throw new Error('Error fetching data');
        }

        //Devolvemos los datos obtenidos
        return response.json();
      })

      //Controlamos los datos
      .then(data => {
        //Almacenamos en una variable las referencias válidas
        const referencias_validas = data.validReferencias;
        referencia_componente = referencias_validas;

        console.log("REFES -> ", referencias_validas)

        //En caso de que no haya referencias válidas
        if (referencias_validas.length === 0) {
          //Llamamos a la función para disponer una alerta para informar al usuario
          mostrarAlerta('Error', 'No hay referencias válidas', 'error', null);

          //Cerramos el modal principal
          $('#modal').modal('hide');

          //Paramos la propagación
          return;
        }

        subirEtapa(id_puesto, operacion, num_picadas, opcion);
      });
    });
  }
}
```

Para añadir una etapa, se genera el modal con el formulario para que el usuario introduzca los datos necesarios y se validan. Después, estos datos se envían otra función para crear la etapa. Es una función reutilizable, por lo que facilita la integración con otras funcionalidades del sistema.

Para subir una etapa, se formatean las referencias para que el backend pueda procesarlas y así obtener el número de embalajes de cada una. Después, se llama a una función que añadirá las etapas a la base de datos, y tras eso, se mostrará una alerta indicando al usuario si desea añadir la distancia recorrida en esa etapa. Una vez finalizado este proceso, la ventana se recargará de forma automática.

```
function anyadirEtapaFinal(id_puesto, operacion_seleccionada, num_picadas) {
  //Serializamos el diccionario con las referencias y el número de embalajes
  referencia_embalaje = encodeURIComponent(JSON.stringify(referencia_embalaje));
  let operacion = encodeURIComponent(operacion_seleccionada);

  if (id_puesto) puestoID = id_puesto;

  if (num_picadas) numero_picadas = num_picadas;

  fetch(`/app/api/anyadirEtapa/${puestoID}/${referencia_embalaje}/${operacion}/${numero_picadas}`, {
    method: "POST"
  });
}
```

Para añadir una etapa a la base de datos, hay que enviar los datos procesados de la nueva etapa al backend.

- Visualización de gráficos: se generan gráficos interactivos, de barras, chimenea y donut, que representan la saturación de los puestos y las actividades realizadas en cada etapa.

```
function gestionarGraficoChimenea(data) {
  //Creamos una variable para almacenar el número de peticiones completadas
  let peticionesCompletadas = 0;

  //Almacenamos en una variable el número de puestos disponibles
  const totalPuestos = data.length;

  //Iteramos por los datos del puesto
  data.forEach(item => {
    //Preparamos la petición GET para obtener los tiempos para generar el gráfico de chimenea
    fetch(`/app/api/graficoChimenea/${item.id}`, { method: "GET" })
    //Controlamos la respuesta
    .then(response => {
      //En caso de que sea mala
      if (!response.ok) throw new Error('Error fetching data');
      //Devolvemos los datos
      return response.json();
    })
    //Controlamos los datos
    .then(data2 => {
      //Iteramos por los datos obtenidos
      data2.forEach(itemChimenea => {
        //Almacenamos en el array los datos que necesitamos para generar el gráfico de chimenea
        conteoGraficoChimenea.push({
          id: item.id,
          nombre: itemChimenea.nombre,
          color: itemChimenea.color,
          minutos: itemChimenea.minutos,
        });
      });

      //Almacenamos el array por el ID del puesto
      conteoGraficoChimenea.sort((a, b) => a.id - b.id);

      //Aumentamos el contador de las peticiones
      peticionesCompletadas++;

      //En caso de que no haya peticiones
      if (peticionesCompletadas === totalPuestos) {
        //Establecemos a true la información
        peticionesFinalizadas.graficoChimenea = true;

        //Llamamos al método para controlar los datos para generar los gráficos
        verificarFinalizacionDeDatos(data2);
      }
    })
  });

  //Controlamos el error
  .catch(error => console.error('Error al cargar datos:', error));
}
```

Para obtener los datos necesarios que se necesitan a la hora de generar el gráfico de chimenea, se realiza una petición al backend.

```
function renderizarGrafico() {
  //Contenedor de los gráficos
  const graficosContainer = document.getElementById('graficos-container');
  graficosContainer.innerHTML = '';

  //Configuramos los diferentes contenedores de los diferentes gráficos
  [...]

  //Gráfico chimenea
  const chartChimenea = new Chart(ctxChimenea, {
    type: 'bar',
    data: {
      labels: ['Label1'],
      datasets: []
    },
    options: {
      responsive: true,
      plugins: {
        legend: {
          labels: {
            color: 'black',
            font: {
              size: 11
            },
            textAlign: 'center',
            boxWidth: 20
          },
          display: false
        },
        tooltip: {
          callbacks: {
            label: context => `${context.dataset.label}: ${context.raw}%`
          }
        }
      }
    }
  });

  //Configuramos los diferentes datos de los gráficos
  [...]

  //Gráfico principal
  const chartBarras = new Chart(ctxBarras, {
    type: 'bar',
    data: {
      labels: nombre_puestos,
      datasets: [{
        label: 'Puestos',
        data: conteos_controlados,
      }]
    },
    options: {
      responsive: true,
      maintainAspectRatio: false,
      plugins: {
        legend: {
          labels: {
            color: 'black',
            textAlign: 'center',
            generateLabels: (chart) => {
              const originalLabels = Chart.defaults.plugins.legend.labels.generateLabels(chart);
            }
          },
        },
      }
    },
    //Función para el clic en las barras o cerca de labels
    onClick: (event, elements) => {
      [...]
    }
  });

  //En caso de que el puesto exista
  if (chartPuesto) {
    //Lo destruimos
    chartPuesto.destroy();
  }

  //Configuramos el gráfico del puesto
  chartPuesto = new Chart(ctxPuesto, {
    type: 'doughnut',
    data: {
      datasets: [{
        data: [0, 0],
        backgroundColor: ['rgba(75, 192, 192, 0.7)', 'rgba(211, 211, 211, 0.3)'],
        borderWidth: 1
      }]
    },
    options: {
      plugins: {
        tooltip: {
          callbacks: {
            label: context => `${context.dataset.data[context.dataIndex]}%`
          }
        },
      },
      responsive: false,
      cutout: '70%',
      rotation: -90
    },
    plugins: [centerTextPlugin]
  });
}
```

Esta función genera directamente gráficos representan la saturación de los puestos y las actividades realizadas en cada etapa.

```
function actualizarGraficoPuesto(chartPuesto, chartChimenea, nombre, conteo, id_puesto, datasets) {
  //Comprobamos si los gráficos del puesto estan bien instanciados
  if (!chartPuesto || !chartChimenea) return;

  //Almacenamos en una variables los datos depurados
  const datasets_depurados = datasets.map(dataset => ({
    ...dataset,
    data: dataset.data.map(d => (typeof d === 'number' && !isNaN(d)) ? d : 0)
  }));

  //Almacenamos en una variable los datos del puesto en cuestión
  const datos_filtrados = datasets_depurados.filter(data => data.id_puesto === id_puesto);

  //Almacenamos en variable los datos
  const valores = datos_filtrados.map(data => data.data[0]);

  //variables para establecer la información de la saturación
  const conteo_libre = conteo > 100 ? 0 : 100 - conteo;

  //Actualizamos el gráfico donut
  chartPuesto.data.labels = ['Tiempo Utilizado', 'Tiempo Libre'];
  chartPuesto.data.datasets[0].data = [conteo, conteo_libre.toFixed(2)];
  chartPuesto.data.datasets[0].backgroundColor = ['rgba(231, 76, 60, 0.7)', 'rgba(46, 204, 113, 0.7)'];
  chartPuesto.data.datasets[0].borderColor = '#5b5b5b';
  chartPuesto.options.plugins.centerText.text = `${conteo}%`;
  chartPuesto.update();

  //Almacenamos en variables los datos necesarios
  const total = 442;
  const porcentajes = valores.map(item => ((item / total) * 100).toFixed(2));

  //Actualizamos el gráfico de chimenea
  chartChimenea.data.labels = ['Actividades'];
  chartChimenea.data.datasets = datos_filtrados.map((data, index) => ({
    label: data.label,
    data: [porcentajes[index]],
    backgroundColor: data.color,
    borderColor: data.color.replace('0.6', '1'),
    borderWidth: 1.5
  }));

  chartChimenea.update();
}
```

Para actualizar dinámicamente los gráficos en función de los datos seleccionados, basta con recoger dicha información del componente seleccionado, y esta función se encargará de redibujar los gráficos automáticamente con los nuevos valores.

Fichero 'index.js'

Este es el núcleo del backend de la aplicación. Define las rutas y la lógica necesaria para interactuar con la base de datos MySQL, procesar solicitudes HTTP y proporcionar datos al frontend. Está diseñado para manejar funcionalidades clave como la gestión de puestos, etapas, referencias, y la visualización de datos mediante gráficos. Además, incluye validaciones y manejo de errores para garantizar la integridad de los datos y la estabilidad del sistema.

- Gestión de puestos: se puede obtener toda la información relevante sobre los puestos, así como eliminarlos.

```
router.get('/obtenerPuestos', (req, res) => {
  //Creamos la conexión a la base de datos
  getDBConnection((err, connection) => {
    //En caso de que se produzca un error...
    if (err) {
      return res.status(400).send('Error al conectar con la base de datos');
    }

    //Almacenamos en una variable la consulta SQL para obtener los puestos disponibles
    const query = `
      SELECT
        DISTINCT(p.id), p.numero, p.nombre, p.numero_operarios,
        ROUND((COALESCE(SUM(e.actividad_minutos_picadas), 0) / 442) * 100, 2) AS saturacion,
        t.turno
      FROM
        puestos p
      INNER JOIN
        turnos t
      ON
        p.id_turno = t.id
      LEFT JOIN
        etapas e
      ON
        p.id = e.id_puesto
      GROUP BY
        p.id,
        p.nombre
      ORDER BY
        p.numero ASC
    `;

    //Ejecutamos la consulta
    connection.query(query, [], (err, results) => {
      //Liberamos la conexión
      connection.release();

      //En caso de que se produzca un error...
      if (err) {
        console.error("> Error: ", err);
        return res.status(500).send('Error en la consulta');
      }

      //En cualquier otro caso...
      console.log("> Resultados PUESTOS: ", results);

      //Enviamos la información y el estado
      return res.status(200).json(results);
    });
  });
});
```

Para obtener la información necesaria de los puestos y visualizarla en la pantalla principal de la aplicación, se ejecuta una consulta SQL que realiza un cruce entre las tablas relacionadas, permitiendo recuperar todos los datos asociados a cada puesto, como las etapas vinculadas, operarios asignados, turno y planta.

```
router.post('/anyadirPuesto/:numero_puesto/:nombre_puesto/:numero_operarios/:turno', (req, res) => {
  //Almacenamos los valores del formulario
  const { numero_puesto, nombre_puesto, numero_operarios, turno } = req.params;

  //Controlamos los valores de los campos necesarios
  if (!numero_puesto || !nombre_puesto || !numero_operarios) {
    return res.status(400).send('Faltan campos en la solicitud');
  }

  //Creamos la conexión a la base de datos
  getDBConnection((err, connection) => {
    if (err) {
      //En caso de que se produzca un error...
      console.error('> Error al conectar a la base de datos: ', err);
      return res.status(500).send('Error al conectar a la base de datos');
    }

    //Almacenamos en una variable la consulta SQL para obtener el ID del turno usando el turno y la planta
    const queryTurno = `
      SELECT
        t.id
      FROM
        turnos t
      WHERE
        t.turno = ?
    `;

    //Ejecutamos la consulta para obtener el ID del turno
    connection.query(queryTurno, [turno], (errorTurno, resultTurno) => {
      if (errorTurno) {
        console.error('> Error a la hora de obtener el ID del turno: ', errorTurno);
        return res.status(500).send('Error al obtener el ID del turno');
      }

      if (resultTurno.length === 0) {
        return res.status(404).send('No se encontró el turno especificado');
      }

      //Almacenamos en la variable el ID del turno
      const id_turno = resultTurno[0].id;

      // Verificamos si el numero_puesto ya existe
      const queryVerificarPuesto = `SELECT COUNT(*) AS count FROM puestos WHERE numero = ?`;

      connection.query(queryVerificarPuesto, [numero_puesto], (errorVerificar, resultVerificar) => {
        if (errorVerificar) {
          console.error('> Error al verificar si el número de puesto ya existe: ', errorVerificar);
          return res.status(500).send('Error al verificar si el número de puesto ya existe');
        }

        // Si el número de puesto ya existe, desplazamos los demás números de puesto
        if (resultVerificar[0].count > 0) {
          // Desplazamos los puestos con número mayor o igual al nuevo
          const queryDesplazar = `UPDATE puestos SET numero = numero + 1 WHERE numero >= ?`;

          connection.query(queryDesplazar, [numero_puesto], (errorDesplazar) => {
            if (errorDesplazar) {
              console.error('> Error al desplazar los puestos: ', errorDesplazar);
              return res.status(500).send('Error al desplazar los puestos');
            }

            // Ahora insertamos el nuevo puesto
            insertarPuesto();
          });
        } else {
          // Si el número de puesto no existe, lo insertamos directamente
          insertarPuesto();
        }
      });
    });

    function insertarPuesto() {
      //Almacenamos en una nueva variable la consulta SQL para añadir el puesto
      const queryPuesto = `
        INSERT INTO
          puestos(numero, nombre, numero_operarios, id_turno)
        VALUES
          (?, ?, ?, ?)
      `;

      //Ejecutamos la consulta para añadir el puesto
      connection.query(queryPuesto, [numero_puesto, nombre_puesto, numero_operarios, id_turno], (errorPuesto, resultPuesto) => {
        if (errorPuesto) {
          console.error('> Error a la hora de añadir el puesto: ', errorPuesto);
          return res.status(500).send('Error a la hora de añadir el puesto');
        }

        //Enviamos el status
        res.status(201).send('Puesto añadido');
      });
    }
  });
});
```

Permite añadir un nuevo puesto a la base de datos. Si el número del puesto ya existe, desplaza los números de los puestos existentes para evitar conflictos y así ordenarlos como el usuario decida.

- **Gestión de etapas:** se puede añadir nuevas etapas asociadas a un puesto, actualizar valores como distancia o número de picadas y obtener etapas agrupadas por puesto o por operación.


```
router.post('/anyadirEtapa/:puesto_id/:referencia_embalaje/:operacion_seleccionada/:numero_picadas', async (req, res) => {
  try {
    //Almacenamos los parámetros de la URL
    const { puesto_id, operacion_seleccionada, numero_picadas } = req.params;

    //Decodificamos y parseamos el JSON de referencia_embalaje con manejo de errores
    let referencia_embalaje;

    try {
      referencia_embalaje = JSON.parse(req.params.referencia_embalaje);
      console.log("> Referencia embalajes: ", referencia_embalaje);
    } catch (err) {
      return res.status(400).send('Error en el formato de referencia_embalaje', err);
    }

    //Obtenemos la query
    const query = queryOperacionSeleccionada(operacion_seleccionada);

    console.log("> Query AÑADIR: ", query);
    if (!query) {
      return res.status(400).send('Operación no válida');
    }

    //Creamos la conexión a la base de datos
    const connection = await new Promise((resolve, reject) => {
      getDBConnection((err, conn) => {
        if (err) reject(err);
        else resolve(conn);
      });
    });

    //Iteramos sobre las referencias y ejecutamos las operaciones
    const keys = Object.keys(referencia_embalaje);
    for (const referencia of keys) {
      const cantidad_mover = referencia_embalaje[referencia];

      const data = [
        puesto_id,
        referencia,
        cantidad_mover,
        operacion_seleccionada,
        numero_picadas
      ];

      //Llamamos a la función para añadir la etapa
      anyadirEtapa_Operacion(connection, query, data, operacion_seleccionada);
    }

    //Liberamos la conexión después de finalizar todas las operaciones
    connection.release();

    //Enviamos la respuesta al cliente
    return res.status(201).send("End point procesado correctamente");
  } catch (err) {
    console.error("> Error en el procesamiento: ", err);
    return res.status(500).send('Error interno del servidor');
  }
});
```

Añade una nueva etapa asociada a un puesto. Procesa las referencias y calcula los valores necesarios antes de insertar la etapa en la base de datos.

```
function queryOperacionSeleccionada(operacion_seleccionada) {
  //Creamos una variable para almacenar la query
  let query;

  //Creamos un switch para controlar la operación seleccionada
  switch (operacion_seleccionada) {
    //En caso de que sea "1. Descarga camión en muelle"
    case '1. Descarga camión en muelle':
      query = `
        INSERT INTO
          etapas (id_puesto, referencia_componente, cantidad_mover, operacion, numero_picadas, DC13, CDC, DS10, CDL, actividad_minutos, actividad_minutos_picadas)
        VALUES
          (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
      `;
      break;

    //Resto de casos
    [...]
```

Se obtiene la consulta para añadir la etapa, dependiendo de la operación seleccionada, ya que cada una de estas contiene diferentes campos de la tabla 'etapas'.

```
function anyadirEtapa_Operacion(connection, query, data, operacion_seleccionada) {
  //Creamos una variable
  let cantidad_mover = data[2], numero_picadas = data[4];

  //Controlamos el tipo de operación
  switch (operacion_seleccionada) {
    //En caso de que sea "1. Descarga camión en muelle"
    case '1. Descarga camión en muelle':
      data.push(
        ((42 * cantidad_mover) / 100), // DC113
        ((6 * cantidad_mover) / 100), // CDC
        ((19 * cantidad_mover) / 100), // DS10
        ((6 * cantidad_mover) / 100), // CDL
        (
          (42 * cantidad_mover) / 100 +
          (6 * cantidad_mover) / 100 +
          (19 * cantidad_mover) / 100 +
          (6 * cantidad_mover) / 100
        ), // Actividad en minutos
        (
          (
            (42 * cantidad_mover) / 100 +
            (6 * cantidad_mover) / 100 +
            (19 * cantidad_mover) / 100 +
            (6 * cantidad_mover) / 100
          ) / numero_picadas
        ) // Actividad en minutos X picada
      );
      break;
    //Resto de casos
    [...]
  }

  //Ejecutamos la sentencia
  connection.query(query, data, (error, result) => {
    //En caso de que ocurra algún error...
    if (error) {
      console.error("> Error a la hora de añadir la operación: ", error);
    } else {
      console.log("> Resultados: ", result);
    }
  });
}
```

Para añadir los datos de las operaciones de la etapa creada, es necesario calcular métricas específicas (como actividad en minutos y actividad por picada) basadas en la operación seleccionada, así como adaptar los cálculos y los datos insertados según el tipo de operación.

```
router.put('/actualizarEtapa/:id_puesto/:operacion/:nuevo_valor/:opcion', (req, res) => {
  //Lógica para actualizar los diferentes datos de las etapas
  [...];
});
```

Para actualizar los distintos atributos de una etapa, concretamente el número de picadas o la distancia, el backend determina qué campo actualizar en función del parámetro '*opcion*' recibido desde el frontend. Primero se realiza una consulta para obtener todas las etapas que pertenecen al puesto y operación indicados. A continuación, se recorre cada una de esas etapas y se ejecuta la actualización correspondiente según la opción seleccionada, ya sea modificar el número de picadas o la distancia.

```
router.get('/obtenerEtapasAgrupadasPuesto/:id_puesto', (req, res) => {
  //Lógica para obtener las etapas agrupadas por puesto (etapas globales)
  [...];
});
```

Se obtiene las etapas asociadas a un puesto, agrupadas por operación y calcula métricas como la cantidad total a mover y la actividad en minutos.

```
router.get('/obtenerEtapasPuesto/:id_puesto/:nombre_etapa', (req, res) => {  
  //Lógica para obtener las etapas de cada puesto  
  [...]  
});
```

Se obtiene las etapas asociadas de cada puesto, para poder disponerlas en las tablas por etapa.

- Gestión de referencias: se pueden validar las referencias, así como obtener las referencias disponibles de un puesto en concreto.

```
router.get('/comprobarReferencias/:referencias', (req, res) => {  
  //Lógica para validar referencias  
  [...]  
});
```

Se comprueba que las referencias introducidas por el usuario están disponibles en la base de datos.

```
router.get('/obtenerReferencias/:id_puesto', (req, res) => {  
  //Lógica para obtener referencias disponibles  
  [...]  
});
```

Se obtienen las referencias disponibles para un puesto en concreto, teniendo en cuenta el turno del mismo para las operaciones, de esta forma, las referencias se disponen en el modal del buscador de referencias.

- Visualización de gráficos: se pueden obtener datos para los gráficos de saturación y gráficos de chimenea.

```
router.get('/obtenerPuestos', (req, res) => {  
  //Consulta SQL para calcular la saturación de los puestos  
  [...]  
});
```

Se obtienen todos los puestos junto con su saturación para disponerlos en el gráfico de barras de la pantalla de inicio.

```
router.get('/graficoChimenea/:id_puesto', (req, res) => {  
  //Consulta SQL para obtener datos del gráfico de chimenea  
  [...]  
});
```

Se obtienen los datos de la saturación de cada etapa de un puesto, así como otros datos importantes para su disposición en el frontend, como son el color o el orden.

- Operaciones generales: contienen rutas tanto para la eliminación de los diferentes registros, como la obtención de datos como los embalajes o los turnos.

```
router.delete('/eliminarRegistro/:id_elemento/:tabla/:id_puesto', (req, res) => {
  /** Almacenamos las variables de los parámetros */
  const { id_elemento, tabla, id_puesto } = req.params;

  //Creamos la conexión a la base de datos
  getDBConnection((err, connection) => {
    let control = Number(id_elemento), query, array_argumetos;

    if (tabla === "etapas") {
      if (!isNaN(control)) {
        query = `
          DELETE
          FROM
            ${tabla}
          WHERE
            id = ?
        `;
        array_argumetos = [id_elemento];
      } else if (isNaN(control)) {
        query = `
          DELETE
          FROM
            ${tabla}
          WHERE
            operacion = ?
            AND
            id_puesto = ?
        `;
        array_argumetos = [id_elemento, id_puesto]
      }
    } else if (tabla === "puestos") {
      query = `
        DELETE
        FROM
          ${tabla}
        WHERE
          id = ?
      `;
      array_argumetos = [id_elemento];
    }

    //Ejecutamos la consulta
    connection.query(query, array_argumetos, (error, results) => {
      console.log("ELIMINAR:", connection.format(query, array_argumetos))
      //En caso de que falle
      if (error) {
        console.error("> Error: ", error);

        //Enviamos el status
        return res.status(501).send("Error en la consulta");
      }

      if (tabla === "puestos") {
        const query = `
          DELETE
          FROM
            etapas
          WHERE
            id_puesto = ?
        `;

        //Ejecutamos la consulta
        connection.query(query, [id_elemento], (error, results) => {
          //En caso de que falle
          if (error) {
            console.error("> Error: ", error);
          }
        })
      }
    })
    //Liberamos la conexión
    connection.release();

    //Enviamos el status
    return res.status(201).send("Registro eliminado");
  });
});
```

Para eliminar diferentes registros de la base de datos, ya sean etapas o puestos, se reciben diferentes parámetros y según esos valores, se construye una consulta SQL para eliminar el registro correspondiente. Para eliminar una etapa, se puede eliminar una etapa especificada por su id o todas las etapas asociadas a una operación y un puesto. Para eliminar un puesto, se elimina por su id y, además, se eliminan todas las etapas asociadas a ese puesto para mantener la consistencia de los datos.

```
router.get('/obtenerTurno/:puesto_id', (req, res) => {  
  //Consulta SQL para obtener el turno de un puesto  
  [...]  
});
```

Se obtiene el turno asociado a un puesto, incluyendo la jornada de inicio y fin, para disponer los datos del turno en el panel de opciones.

```
router.get('/obtenerEmbalajes/:referencia/:puesto_id', (req, res) => {  
  //Consulta SQL para obtener datos de embalajes  
  [...]  
});
```

Se obtiene la cantidad de embalajes asociados a una referencia y un puesto específico para poder añadir una etapa a un puesto.

Fichero 'app.ejs'

Este es la plantilla principal del frontend de la aplicación. Define la estructura de la interfaz de usuario, incluyendo el diseño del panel lateral, los botones de interacción, los gráficos y las tablas que muestran información sobre los puestos y etapas. El propósito principal de este archivo es proporcionar una interfaz interactiva y visualmente atractiva para que los usuarios puedan gestionar los puestos y etapas, visualizar gráficos y realizar acciones como añadir o eliminar registros.

```
<div class="flex space-x-4">  
  <!-- Botón para añadir un puesto -->  
  <button id="anyadirPuesto" class="...">  
    <span><i class="bi bi-person-workspace"></i> Añadir puesto</span>  
  </button>  
  
  <!-- Botón para añadir una etapa -->  
  <button id="anyadirEtapa" style="display: none;" class="...">  
    <span><i class="bi bi-plus-square"></i> Añadir etapa</span>  
  </button>  
</div>
```

El panel de opciones proporciona botones para añadir puestos y etapas, así como secciones informativas como la leyenda de colores y el turno de la etapa. El botón "Añadir etapa" está oculto inicialmente y se muestra dinámicamente según las acciones del usuario.

```
<div id="graficos-container" class="..." style="height: 500px; width: 100%;">
  <!-- El gráfico se renderizará aquí -->
</div>
```

Para renderizar gráficos generados dinámicamente por el frontend utilizando Chart.js, como gráficos de saturación de puestos, se utiliza este contenedor vacío.

```
<div id="tablaEtapasGlobal"></div>
```

Para mostrar la tabla dinámica con las etapas disponibles, se define un contenedor vacío donde se renderizan, el cual se actualiza dinámicamente mediante JavaScript según las acciones del usuario.

6.4.3. Cuestiones de diseño e implementación reseñables

Durante el desarrollo de la aplicación se han tenido en cuenta diversas decisiones clave tanto a nivel de diseño como de implementación, con el objetivo de lograr un sistema eficiente, intuitivo y escalable. Uno de los aspectos más destacables ha sido la organización lógica de los datos, en la que todas las etapas deben estar siempre asociadas a un puesto concreto. Esta decisión condiciona muchas de las acciones del usuario, como la creación o visualización de etapas, lo que garantiza la coherencia estructural del sistema.

A nivel de implementación, se ha buscado reducir la duplicación de código mediante funciones reutilizables, como es el caso de la función que permite eliminar tanto puestos como etapas. Esta unificación permite centralizar la lógica de eliminación, simplificar el mantenimiento del sistema y mejorar la legibilidad del código.

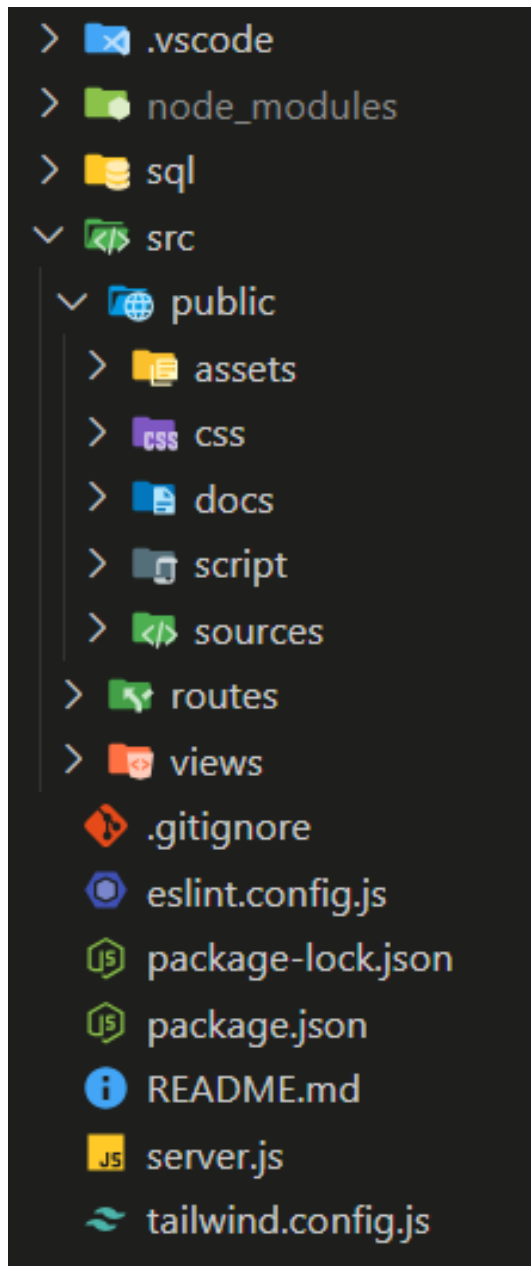
Otro aspecto destacable ha sido la actualización dinámica de los gráficos. En lugar de recargar toda la página, se realiza una petición específica con los datos seleccionados, lo que permite modificar el contenido de los gráficos de forma inmediata, ofreciendo una experiencia de usuario más fluida.

Asimismo, se ha optado por almacenar colores representativos para cada etapa directamente en la base de datos, lo que permite mantener una consistencia visual entre las tablas y los gráficos sin depender exclusivamente del frontend.

En cuanto a las vistas, se ha utilizado EJS para renderizar contenido dinámico directamente desde el servidor, facilitando la integración con los datos obtenidos desde MySQL y permitiendo mantener una estructura clara entre lógica de servidor y presentación.

La modularidad del código también ha sido una prioridad, separando de forma lógica los distintos módulos del backend y del frontend, permitiendo una navegación clara por los archivos del sistema y facilitando su mantenimiento y escalabilidad.

A continuación, se muestra una imagen de la estructura del proyecto:



6.5. Pruebas

A continuación, se detallan las pruebas realizadas:

Caso de uso	Prueba realizada	Resultado
Añadir puesto	El usuario introduce los datos del puesto y pulsa el botón "Añadir puesto".	El puesto se añadió y el gráfico principal se actualizó.
Añadir etapa	El usuario selecciona un puesto, pulsa el botón "Añadir etapa", introduce los datos de la nueva etapa y confirma.	La etapa se añadió y los gráficos se actualizaron correctamente.
Eliminar puesto	El usuario selecciona un puesto y pulsa el botón rojo con el icono de papelera.	El puesto y todas sus etapas se eliminaron.
Eliminar etapa	El usuario selecciona una etapa y pulsa el botón rojo con el icono de papelera.	La etapa se eliminó y la tabla y los gráficos se actualizaron.
Editar distancia de una etapa	El usuario selecciona una etapa, pulsa el icono de un plano blanco, introduce una distancia y confirma.	La distancia y las tablas y gráficos se actualizaron.
Editar picadas de una etapa	El usuario selecciona una etapa, pulsa el icono de información gris, selecciona un número y confirma.	Las picadas, los tiempos y las tablas y gráficos se actualizaron.
Buscar referencias	El usuario selecciona un puesto, pulsa el botón de "Añadir etapa" y pulsa el buscador del cuadro de texto de las referencias.	Las referencias, filtradas por turno se disponen en el modal.
Visualizar gráficos de saturación	El usuario accede al panel principal.	Los gráficos de saturación se generan y se muestran en el panel correctamente.
Visualizar etapas agrupadas por puesto	El usuario selecciona un puesto del gráfico principal.	Las etapas se mostraron correctamente agrupadas en una tabla dinámica.
Visualizar datos detallados de las etapas	El usuario selecciona una etapa de un puesto.	Los datos desglosados de la etapa se mostraron.
Añadir puesto vacío	El usuario intenta añadir un nuevo puesto, pero sin completar todos los campos obligatorios.	Se muestra un mensaje de error indicando que faltan campos obligatorios.

7. MANUALES DE USUARIO

7.1. Manual de usuario

INTERFAZ PRINCIPAL

Al iniciar sesión, se accede a la interfaz principal de la aplicación. Esta pantalla se compone de un panel de opciones situado en el lateral izquierdo, donde se encuentran botones para realizar acciones como añadir nuevos puestos o registrar etapas. A la derecha, se muestra el gráfico principal en el que se visualizan los puestos junto con su nivel de saturación. El botón situado en la esquina superior derecha es un acceso directo a este manual.

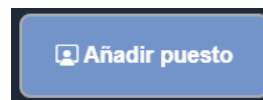


Al pasar el cursor sobre cualquiera de los puestos representados, aparece una etiqueta que muestra el nombre del puesto y su porcentaje de saturación. La leyenda del gráfico utiliza un sistema de colores que varía en función del turno asignado a cada puesto, facilitando así la identificación visual rápida de la información. Junto a la saturación de los puestos, también aparece la media de la saturación de la planta.

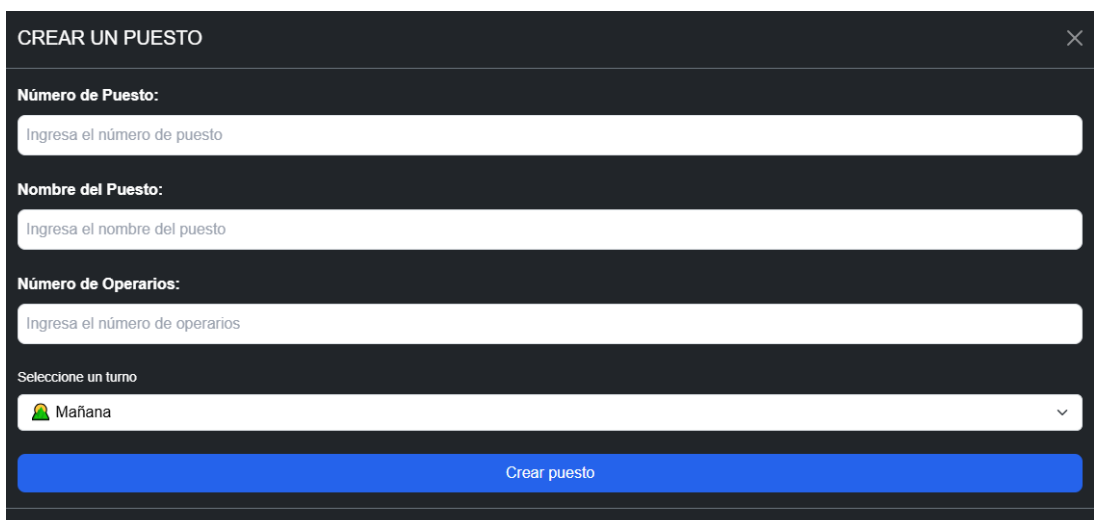
GESTIÓN DE PUESTOS

Añadir un puesto

Para añadir un puesto, pulsar sobre el botón



Después, se abrirá el siguiente modal donde se deberán de completar los campos.



The 'CREAR UN PUESTO' modal form contains the following fields and controls:

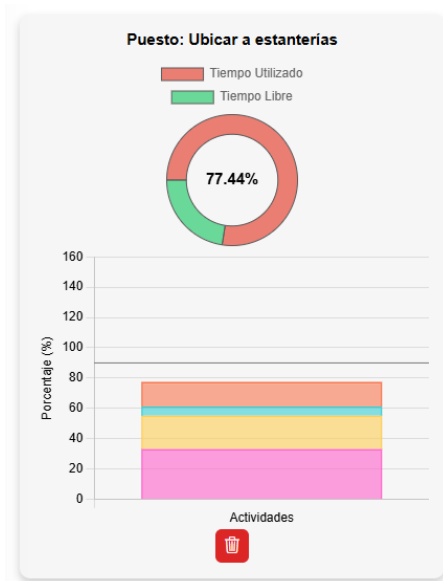
- Número de Puesto:** Input field with placeholder 'Ingresa el número de puesto'.
- Nombre del Puesto:** Input field with placeholder 'Ingresa el nombre del puesto'.
- Número de Operarios:** Input field with placeholder 'Ingresa el número de operarios'.
- Seleccione un turno:** Dropdown menu with 'Mañana' selected.
- Crear puesto:** Blue button at the bottom.

Eliminar un puesto

Para eliminar un puesto, primero se deberá seleccionar y aparecerá un nuevo gráfico, con la saturación desglosada, en el que está el

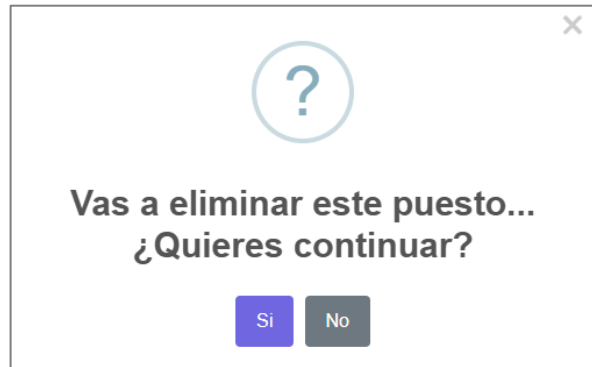


icono de basura.



Pulsar sobre para eliminar el puesto.

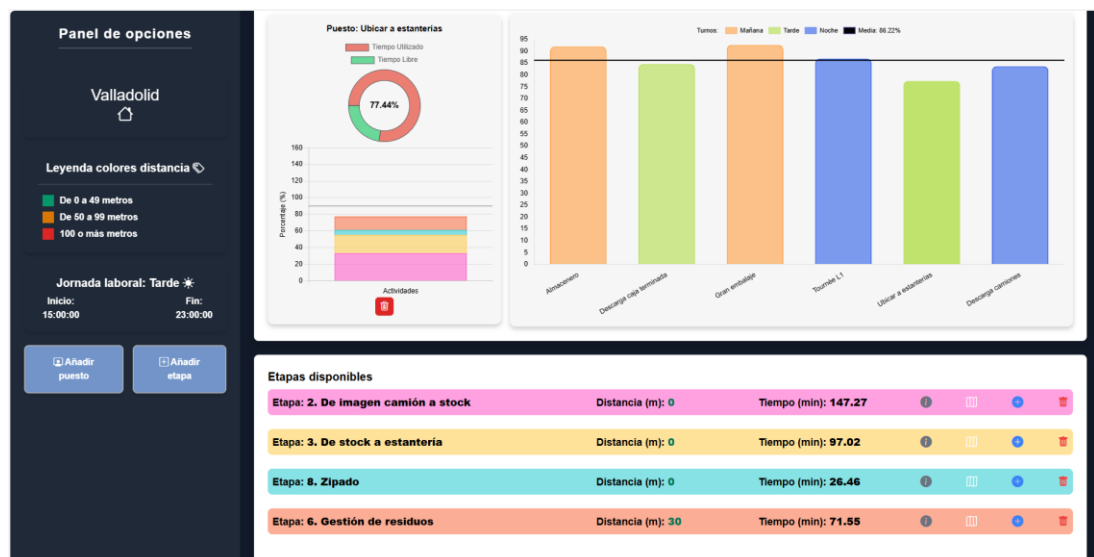
Antes de eliminar el puesto, la aplicación pedirá confirmación al usuario:



Una vez pulsado el botón 'Si', el puesto se eliminará de la base de datos y la página se recargará, actualizándose los gráficos.

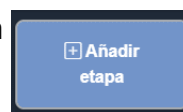
GESTIÓN DE ETAPAS

Las etapas disponibles de un puesto aparecen en el momento en el que se pulsa sobre uno de ellos.



Añadir una etapa

Para añadir una etapa, pulsar sobre el botón



Después, se abrirá el siguiente modal donde se deberán de completar los campos.

Re llene los campos para añadir una etapa

Referencia del componente:

Ingresa la referencia del componente

Número de embalajes a mover en simultáneo: 1 Operación: 1. Descarga camión en muelle

Añadir etapa

Para añadir una etapa, se pueden introducir las referencias de forma manual o buscarlas en la base de datos. Para ello, simplemente pulsar sobre el icono de la lupa al final del cuadro de texto.

Se abrirá un modal con todas las referencias disponibles para ese puesto. Se pueden seleccionar, y para añadirlas a la etapa pulsar sobre el botón

Referencias obtenidas:

Buscar referencia...

Referencias
R100577958
319A62831R
521565789R
384630181R
383424036R
328A06555R
320501560R
306206219R
305703721R
240M71003R
8200204823
319183504R
8200142677
384697081R
290T57798R
240M77176R

Guardar selección

El número de embalajes a mover en simultáneo y la operación son desplegables. En ellos seleccionar los datos que se deseen. Una vez introducidos y seleccionados todos los datos, pulsar sobre “Añadir etapa” para agregar la etapa a la base de datos y a ese puesto en concreto. Todos los gráficos se actualizarán automáticamente al realizar esta acción.

Si la operación seleccionada ya existe en ese puesto, simplemente las referencias introducidas se añaden a esa etapa, si no, se crea una nueva.

También se pueden añadir etapas pulsando este icono , que se encuentra en las etapas globales. Se abrirá el mismo modal, solo que el número de picadas y la operación ya aparecen seleccionadas por defecto, con los mismos valores de la etapa desde la que se accedió.

Visualizar etapas

Para visualizar las etapas de referencia (subetapas), pulsar sobre la tarjeta de la etapa que se desea consultar.

Panel de opciones

Valladolid

Legenda colores distancia

- De 0 a 49 metros
- De 50 a 99 metros
- 100 o más metros

Jornada laboral: Tarde

Inicio: 15:00:00 Fin: 23:00:00

Añadir puesto Añadir etapa

Etapa: 2. De imagen camión a stock Distancia (m): 0 Tiempo (min): 147.27

Etapa: 2. De imagen camión a stock	Componente: 319A62831R	8.80
Etapa: 2. De imagen camión a stock	Componente: 305703721R	2.64
Etapa: 2. De imagen camión a stock	Componente: 328A06555R	11.73
Etapa: 2. De imagen camión a stock	Componente: 384630181R	13.79
Etapa: 2. De imagen camión a stock	Componente: 240M71003R	9.39
Etapa: 2. De imagen camión a stock	Componente: 8200204823	15.25
Etapa: 2. De imagen camión a stock	Componente: 8200142677	5.87
Etapa: 2. De imagen camión a stock	Componente: 290T54963R	19.95
Etapa: 2. De imagen camión a stock	Componente: 7700866290	7.63
Etapa: 2. De imagen camión a stock	Componente: 290T57798R	7.63
Etapa: 2. De imagen camión a stock	Componente: 240M77176R	3.52
Etapa: 2. De imagen camión a stock	Componente: 290H98512R	23.47
Etapa: 2. De imagen camión a stock	Componente: 328A00952R	17.60

Para acceder a la información detallada de cada subetapa, pulsar sobre la tarjeta de la subetapa a consultar. Se desplegará la siguiente tabla con información:

Etapa: 2. De imagen camión a stock		Distancia (m): 0	Tiempo (min): 147.27		
Etapa: 2. De imagen camión a stock		Componente: 319A62831R		8.80	
Operación	Símbolo (estándar MTM3)	Tiempo MTM3 (CTMin)			
TOMA SIMPLE en el stock de embalaje lleno (altura media)	PS14	38		30	11.40
CAMBIO de dirección en carga al llegar a stock	CDC	6		30	1.80
DEPOSITA SIMPLE en stock de proximidad (altura media)	DS14	38		30	11.40
CAMBIO de dirección libre al llegar a imagen camión	CDL	6		30	1.80
Distancia	Metros 0	Velocidad 0.6		30	0.00
Actividad total en minutos					26.40
Actividad en minutos (según el número de picadas simultáneas)					8.80
Número de picadas					3

Esta tabla contiene información importante a tener en cuenta, como las operaciones que se realizan por cada referencia, con una explicación detallada sobre la operación, acompañada de su símbolo, el tiempo – en centésimas de minuto – que se necesita

para el desarrollo de esa operación. A continuación, en la tabla, aparece el número de embalajes que se necesitan mover de esa referencia, y por último el tiempo total que ocupa cada operación teniendo en cuenta el tiempo MTM3 y la cantidad de embalajes.

Al final de la tabla, se pueden ver datos relevantes como la distancia que recorre esa etapa, junto con la velocidad a la que se recorre y el número de embalajes. Operando sobre todos esos datos, se obtiene el tiempo total que se emplea en el desplazamiento.

Por último, se puede observar la cantidad total de tiempo, en minutos, que se emplea en realizar esa subetapa, la actividad total en minutos por picada¹ y el número de picadas.

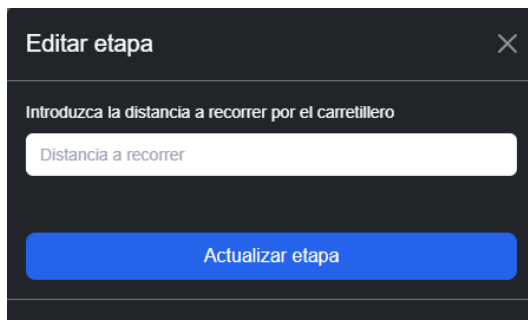
Modificar etapas

Para modificar una etapa, solo se pueden modificar dos valores: la distancia y el número de picadas.

Modificar distancia

Para modificar la distancia de una etapa, pulsar sobre el icono .

Aparecerá el siguiente modal en el que se deberá introducir la distancia a recorrer, en metros:



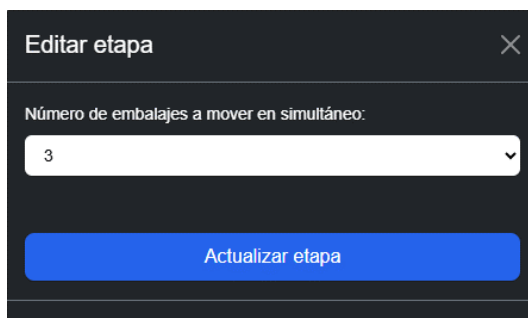
El modal "Editar etapa" tiene un título con un icono de cerrar (X). El contenido principal indica "Introduzca la distancia a recorrer por el carretillero" y contiene un campo de texto con el placeholder "Distancia a recorrer". En la parte inferior hay un botón azul que dice "Actualizar etapa".

Al pulsar sobre el botón "Actualizar etapa", se cerrará el modal y se recargará de forma automática la página, actualizándose así las etapas y puestos.

Modificar número de picadas

Para modificar el número de picadas de una etapa, pulsar sobre el icono .

Aparecerá el siguiente modal en el que se deberá seleccionar el nuevo número de picadas:

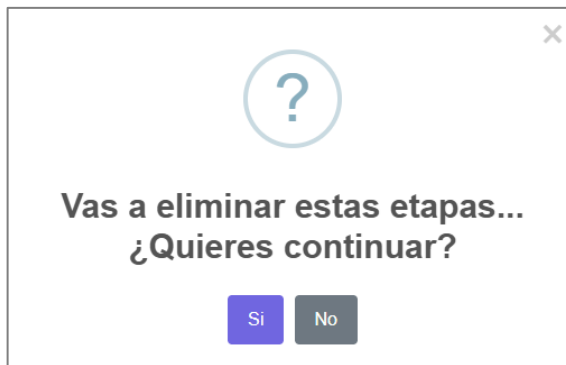


El modal "Editar etapa" tiene un título con un icono de cerrar (X). El contenido principal indica "Número de embalajes a mover en simultáneo:" y contiene un menú desplegable con el valor "3" seleccionado. En la parte inferior hay un botón azul que dice "Actualizar etapa".

Al pulsar sobre el botón "Actualizar etapa", se cerrará el modal y se recargará de forma automática la página, actualizándose así las etapas y puestos.

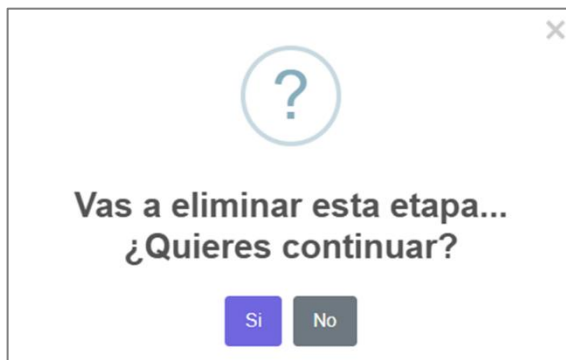
Eliminar etapas

Para eliminar una etapa, pulsar sobre  . Antes de eliminarla, la aplicación pedirá confirmación al usuario:



Una vez pulsado el botón 'Si', las etapas se eliminarán de la base de datos y la página se recargará, actualizándose los gráficos.

Para eliminar una etapa de referencia, pulsar sobre  . Antes de eliminarla, la aplicación pedirá confirmación al usuario:



Una vez pulsado el botón 'Si', la etapa se eliminará de la base de datos y la página se recargará, actualizándose los gráficos.

PREGUNTAS FRECUENTES

1. ¿Qué hacer si no se cargan los datos en la aplicación?

Verifica tu conexión a la red local o a Internet. Intenta recargar la página presionando Ctrl + R o el botón de recarga del navegador. Si el problema persiste, contacta al administrador del sistema.

2. ¿Qué sucede si intento eliminar un puesto con etapas asociadas?

Al eliminar un puesto, todas las etapas asociadas a ese puesto también se eliminarán automáticamente para mantener la consistencia de los datos

3. ¿Cómo puedo visualizar los gráficos de saturación?

Los gráficos de saturación se generan automáticamente al acceder al panel principal. Si no se muestran, verifica que existan datos en la base de datos y recarga la página.

4. ¿Puedo editar una etapa después de haberla añadido?

Sí, pero solo la distancia y el número de picadas. Para ello selecciona el icono de plano, para las distancias, o el de información, para las picadas. Modifica los campos en el modal y guarda los cambios.

Para modificar otros datos que no sean los mencionados anteriormente, elimina la etapa y vuélvela a crear.

5. ¿Qué hacer si no las referencias que he introducido al añadir una etapa no existen?

Estas etapas no se crearán. Asegúrate de que las referencias estén correctamente registradas en la base de datos.

Contacta al administrador para verificar la información de las referencias.

6. ¿Qué navegadores son compatibles con la aplicación?

La aplicación es compatible con los navegadores más recientes, como Google Chrome, Mozilla Firefox y Microsoft Edge.

7. ¿Cómo puedo reportar un problema técnico?

Contacta al administrador del sistema proporcionando una descripción detallada del problema, incluyendo los pasos realizados antes de que ocurriera el error.

8. ¿Qué hacer si necesito añadir una operación nueva?

Las operaciones disponibles están predefinidas en la aplicación. Si necesitas añadir una nueva operación, contacta al administrador del sistema para que realice los ajustes necesarios en el backend.

9. ¿Cómo puedo consultar el turno de un puesto?

Selecciona un puesto en el panel principal. La información del turno (inicio y fin de jornada) se mostrará automáticamente en el panel lateral.

10. ¿Qué sucede si intento añadir un puesto sin completar todos los campos?

La aplicación mostrará un mensaje de error indicando que faltan campos obligatorios. Completa todos los campos requeridos antes de intentar añadir el puesto nuevamente.

7.2. Manual de instalación

Para poder instalar la aplicación son necesarios unos requisitos previos: tener instalado **Node.js** (versión 14 o superior) y **MySQL** (versión 5.7 o superior), así como tener configurado un entorno de línea de comandos (como PowerShell, Bash o Terminal). A continuación, se detallan los pasos a seguir para una correcta y completa instalación de la aplicación:

1. Clonar el repositorio:

```
git clone https://github.com/carlafb-horse/APP_CARRETILLERO.git
cd APP_CARRETILLERO
```

2. Instalar las dependencias:

```
npm install
```

3. Configurar la base de datos:

- Crear una base de datos en MySQL con el nombre *carretillero*.
- Importar el archivo *database.sql* para crear las tablas necesarias:

```
mysql -u root -p carretillero < database.sql
```

- Configurar las credenciales de la base de datos en el código (index.js – línea 12):

```
//Creamos el pool de conexiones
const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'toor',
  database: 'carretillero',
  port: 3306
});
```

4. Iniciar el servidor:

```
npm start
```

5. Acceso a la aplicación:

Tras realizar los pasos detallados anteriormente, la aplicación ya estará preparada para su uso. Para acceder a ella solamente hay que acceder, desde un navegador web a esta URL: <http://localhost:3000>. Una vez ahí, ya se puede utilizar la aplicación con todas sus funcionalidades.

8. CONCLUSIONES Y POSIBLES AMPLIACIONES

El desarrollo de esta aplicación ha permitido crear una herramienta web funcional que facilita la gestión y análisis de la saturación de puestos de trabajo en un entorno industrial. A lo largo del proyecto, se han logrado integrar diferentes tecnologías tanto en el frontend como en el backend, garantizando una experiencia de usuario fluida, una interfaz clara y una estructura de datos coherente.

Durante el proceso se han alcanzado los objetivos propuestos: desde la implementación de la gestión de puestos y etapas, hasta la visualización dinámica mediante gráficos interactivos, sin olvidar la capacidad de registro, edición, eliminación y consulta de datos relevantes. La aplicación resulta útil para optimizar la planificación operativa, identificar cuellos de botella y mejorar la eficiencia en la distribución de tareas.

El diseño modular del sistema y la utilización de tecnologías estándar (Node.js, MySQL, TailwindCSS, EJS, entre otras) facilitan el mantenimiento, la escalabilidad y la posible incorporación de nuevas funcionalidades en el futuro.

Además, este proyecto ha representado un reto académico y técnico que ha permitido aplicar conocimientos adquiridos durante el curso, como la estructuración de bases de datos, diseño de interfaces, programación del lado servidor y cliente, así como adquirir nuevos conocimientos.

A futuro, esta aplicación podría mejorar y expandirse mediante diversas funcionalidades, entre las cuales destacan:

- **Sistema de autenticación de usuarios** con diferentes roles (administrador, operario, supervisor) para mejorar la seguridad y personalizar la experiencia.
- **Importación y exportación de datos** en formatos adicionales, como Excel o integración con otras herramientas ERP.
- **Notificaciones y alertas automáticas**, por ejemplo, si un puesto supera cierto umbral de saturación.
- **Versión responsive optimizada para dispositivos móviles** para facilitar su uso en planta o en movilidad.
- **Panel de administración avanzada** con filtros, estadísticas globales y comparativas entre turnos, plantas o periodos.

Estas ampliaciones permitirían enriquecer aún más la funcionalidad y el alcance de la herramienta, alineándola con las necesidades reales de entornos industriales más complejos y exigentes.

9. BIBLIOGRAFÍA

[Modified Waterfall Model | Download Scientific Diagram](#)

[Ciclo de vida del desarrollo de software: fases y modelos clave](#)

[¿Qué es un diagrama de Gantt?](#)

[Node.js — Ejecuta JavaScript en cualquier parte](#)

[Tutorial. Diseño de bases de datos con MySQL Workbench](#)

[The Modern JavaScript Tutorial](#)

[EJS -- Embedded JavaScript templates](#)

[Documentation - Tailwind CSS](#)

[Express - Node.js web application framework](#)

[Chart.js | Chart.js](#)

[SweetAlert2 - a beautiful, responsive, customizable and accessible \(WAI-ARIA\) replacement for JavaScript's popup boxes](#)

[Bootstrap · The most popular HTML, CSS, and JS library in the world.](#)

10. ANEXOS

10.1. ANEXO I: Código de la aplicación

El código más relevante de la aplicación ya ha sido desarrollado a lo largo de otros apartados de esta memoria.

10.2. ANEXO II: Control de versiones

Se ha creado un repositorio público en GitHub donde se ha subido el proyecto:

https://github.com/carlafb-horse/APP_CARRETILLERO

10.3. ANEXO III: Diagrama de Gantt

El diagrama de Gantt del desarrollo del proyecto es el siguiente:

Nombre de la tarea	Fecha de inicio	Fecha de fin	Horas planificadas	Horas reales	10.03.25	17.03.25	24.03.25	31.03.25	07.04.25	14.04.25	21.04.25	28.04.25	05.05.25	12.05.25	19.05.25
PROYECTO	10.03.25	19.05.25	100	130											
ANÁLISIS Y DISEÑO	10.03.25	24.03.25	15	20											
Definición de requisitos	10.03.25	17.03.25	5	5											
Diseño de la base de datos	17.03.25	24.03.25	5	8											
Diseño de la arquitectura de software	24.03.25	30.03.25	5	7											
CONFIGURACIÓN DEL ENTORNO	17.03.25	30.03.25	10	15											
Inserción de datos en la base de datos	17.03.25	24.03.25	4	8											
Configuración del entorno de desarrollo	24.03.25	30.03.25	2	3											
Inicialización del proyecto backend	24.03.25	30.03.25	4	4											
DESARROLLO DEL PROYECTO	24.03.25	14.04.25	40	50											
Módulo de gestión de puestos	24.03.25	07.04.25	20	25											
Módulo de gestión de etapas	31.03.25	14.04.25	20	25											
PRUEBAS Y DEPURACIÓN	10.03.25	14.04.25	20	25											
Pruebas unitarias	14.04.25	21.04.25	4	5											
Pruebas de integración y de API	21.04.25	05.05.25	8	10											
Depuración de errores y optimización	21.04.25	05.05.25	8	10											
DOCUMENTACIÓN DEL PROYECTO	24.03.25	12.05.25	10	15											
Documentación técnica	24.03.25	05.05.25	6	10											
Documentación de usuario y manual de usuario	12.05.25	18.05.25	4	5											
PRESENTACIÓN Y DESPLIEGUE	10.03.25	14.04.25	5	5											
Preparación de la presentación	19.05.25	22.05.25	3	3											
Despliegue de la aplicación	12.05.25	18.05.25	2	2											